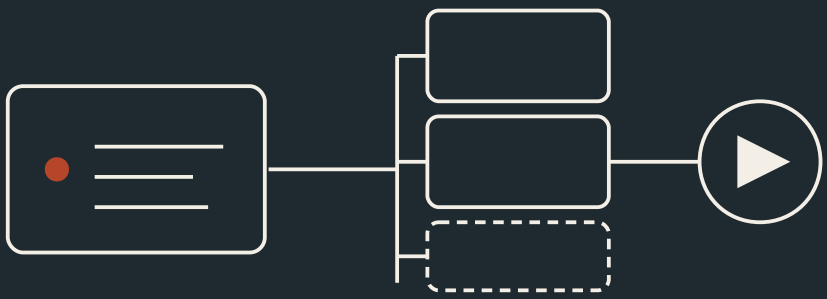
EgOS

Instruction manual

Home media centre: photos, films, music and series on Debian 13.
Raspberry Pi 5 · PC x86-64



Version 4.3.0

2026-11-03

EgOS 4.3.0 — Instruction manual

Generated on 2026-11-03 from `docs/manual/manual.md` and `docs/scripts/*.md` with `build-manual.py`.

Reference: scripts `EgOS4-arm64.sh` and `EgOS4-amd64.sh`.

Contents

PART I — USER GUIDE

Welcome 6
Installation 8
First boot: guided setup 9
Second boot: automatic preparation 10
Everyday use 11
Disks: the storage pool 13
Optional services 15
Command quick reference 16
Troubleshooting 17

PART II — ADMINISTRATOR GUIDE

Architecture 20
Where things are 22
Security and known limits 24
Release notes 26

PART III — TECHNICAL REFERENCE

Scripts and functions 28

Build scripts 28
EgOS4-arm64.sh 28
EgOS4-amd64.sh 32
Builders — building the ISO and the Pi image in throwaway containers 36
live-build hooks (amd64 only) 38

First and second boot 41
setup-webapp.sh 41
webapp.py 42
egos-growroot.sh (arm64 only) 44
deploy-compose.sh 45
dirtree.sh 46
wizardall2.sh 47
Stack configuration (compose, env, JSON) 49
egos-home — home page with one card per container 52

Storage **56**
egos-storage 56
Cockpit page "Storage pool" 69
systemd units, drop-ins and udev rule 72

Optional services **76**
egos-extras 76
torrent_web.py 78
qbt-sort.py 80
audiotags.py 83

PART I

User guide

Welcome

Thank you for choosing **EgOS**, the system that turns a small home computer into a personal media centre: your photos, films, music and series, always at hand on every device in your home network, with no subscriptions and no third-party cloud.

This booklet takes you from installation to everyday use. **Part I** is for the people who use the box; **Part II** is for whoever administers it; **Part III** describes, script by script and function by function, how the system is built.

What EgOS does

SERVICE	WHAT IT IS FOR	ADDRESS
Home page	one card per service, with a link to each	`http://egocenter.local:8000`
Jellyfin	watch films and series, listen to music from TV, phone, browser	`http://egocenter.local:8096`
Immich	backup and gallery for your phone's photos	`http://egocenter.local:2283`
Shared folder (Samba)	copy media files from your PC	`\\egocenter.local\media`
Cockpit	administration console and disk management	`https://egocenter.local:9090`
SSH	terminal access	`ssh user@egocenter.local`

Contents and requirements

EDITION	FOR	INSTALLATION MEDIUM
`egos-4.3.0-amd64.iso`	x86-64 PC, mini PC, virtual machine	USB stick (automatic installation)
`egos-4.3.0-arm64.img`	Raspberry Pi 5 with NVMe SSD	image written directly to the NVMe

You also need:

- an Internet connection over an **Ethernet cable** (needed at the second boot to download the services);
- a phone or a PC for the initial setup;
- optional: one or more external USB disks for more space.

Warnings

> **WARNING** — Installing on a PC **completely erases** the target disk without asking for confirmation. If the computer has several disks, the installer asks which one to use: choose carefully.

> **WARNING** — Adding a disk to the pool **erases everything** on it. EgOS always asks, and wants you to type the disk's name to confirm.

> **NOTE** — EgOS does not use RAID: if a data disk fails you lose the files that were on it, not those on the other disks. Keep a copy of important photos somewhere else.

Installation

On a PC (amd64)

1. Write `egos-4.3.0-amd64.iso` to a USB stick (with `dd`, balenaEtcher, Rufus in DD mode, etc.).
2. Connect the PC to the network with the **Ethernet cable**.
3. Boot the PC from the stick (BIOS or UEFI, either works).
4. The menu opens on **Start installer** and starts by itself after **5 seconds**.
5. The installation is fully automatic (about 5 minutes): Italian language, Italian keyboard, Europe/Rome time zone, machine name `egocenter`.
6. When it is done the PC **powers off**. Remove the stick.

On a Raspberry Pi 5 (arm64)

1. Connect the NVMe SSD to your PC with a USB adapter.
2. Write `egos-4.3.0-arm64.img` to the NVMe with **Raspberry Pi Imager** → *Use custom* (or with `dd` from Linux).
3. Fit the NVMe to the Pi 5. In the EEPROM, the boot order (`BOOT_ORDER`) must include the value `6` (NVMe): check it with `rpi-eeprom-config`.
4. Connect the Ethernet cable and the power supply.

At the first boot the system grows its partition by itself to fill the whole NVMe.

> **TIP** — With a monitor connected, the login screen shows the box's IP address and the useful addresses. Handy when the `.local` name does not work.

First boot: guided setup

At the first boot EgOS serves a small setup page. You can reach it in two ways.

Over Wi-Fi (if the box has Wi-Fi)

1. On your phone, look for the Wi-Fi network **SetupBox** and connect.
2. The network password is `setup1234`.
3. The phone opens the "Sign in to network" page by itself; if it does not, open any `http://` address in the browser.

Over the cable

From a PC on the same network open `http://setupbox.local`, or `http://<box IP>` (shown on the monitor connected to the box, or in your router's page).

Fill in the form

FIELD	USED FOR
Email	Immich administrator account
Username	user for the system, Jellyfin, the shared folder and Cockpit
Password	one password for all the services above

Press **Complete Setup**. "Setup Complete" appears: the box turns off the SetupBox network and **reboots**.

> **WARNING** — During setup the page is not protected: whoever fills it in first becomes the administrator. Do the first boot in a trusted environment and complete it straight away.

Second boot: automatic preparation

After the reboot the box, now reachable as `egocenter.local`, works on its own:

1. it waits until the clock is synchronised and the Internet answers;
2. it creates the folders for films, music, series, ROMs, downloads and photos;
3. it prepares the shared folder with your username and password;
4. it downloads and starts Immich and Jellyfin (a few GB: on a slow line this can take a long time);
5. it creates your administrator account in Immich and Jellyfin;
6. it starts the **home page** `http://egocenter.local:8000`, with a card for every service.

If something goes wrong (for example no Internet), the preparation **retries at the next boot**.

Before you start: create the Jellyfin libraries

Jellyfin is configured in Italian, but the libraries have to be added by hand, once: open Jellyfin → **Dashboard** → **Libraries** → **Add Media Library**.

CONTENT TYPE	FOLDER TO CHOOSE	WHERE TO COPY THE FILES
Movies	`/Films`	`media\Films`
Shows	`/Serie`	`media\SerieTv`
Music	`/Musics`	`media\Musics`
(optional) Other	`/ROMs`	`media\ROMs`

Everyday use

Copying films, music and series

From your PC open the shared folder **media** and sign in with your username and password.

SYSTEM	ADDRESS
Windows	in File Explorer: `\\egocenter.local\media` (the box also appears under **Network**)
macOS	Finder → Go → Connect to Server → `smb://egocenter.local/media`
Linux	`smb://egocenter.local/media` in the file manager

Organise the files like this (these are the names Jellyfin recognises best):

```
Films/Inception (2010)/Inception (2010).mkv
SerieTv/Dark (2017)/Season 01/Dark (2017) - S01E01.mkv
Musics/Artist/Album/01 - Track.flac
```

Jellyfin picks up new files by itself; you can force a scan from **Dashboard → Libraries**.

Watching and listening

Open `http://egocenter.local:8096` in the browser, or install the **Jellyfin** app on your TV, phone or tablet and enter `http://egocenter.local:8096` (or the box's IP) as the server.

Phone photos

Install the **Immich** app, enter `http://egocenter.local:2283` as the server, sign in with the **email and password** from the setup and turn on automatic backup.

The administration console (Cockpit)

Open `https://egocenter.local:9090`. The browser reports an untrusted certificate (it is self-signed): accept the risk to continue. Sign in with your username and password. For disk operations click **Limited access** at the top right and enter your password again.

The **Storage pool** page is described in the next chapter. The **Overview** page shows a red banner only when a pool disk is missing.

The home page

Open `http://egocenter.local:8000`: one card for every service of the box, with a link to it. The page follows Docker by itself: a container that publishes a web page gets a card within 15 seconds (the optional services too, once enabled), a stopped one turns grey, one whose health check fails gets a yellow border. Cockpit always has its card. Databases and other non-web ports never appear.

Disks: the storage pool

EgOS gathers all the space into one big folder, the **pool** (`/srv/egos/pool`). At first the pool lives on the system disk; every disk you add joins the pool **without moving anything and without stopping the services**. Films, photos and the shared folder always see the same paths.

SITUATION	WHERE NEW FILES GO
No external disk	system disk
One or more disks	the disk with the most free space
A disk unplugged	the other disks; that disk's files are not visible until it is back

Each disk needs at least **20 GB free** to receive new files. Once there are data disks, the system disk no longer receives new files: if every data disk drops below 20 GB free, writes fail with a "read-only file system" error. This is intended: the system disk never fills up.

Adding a disk

From Cockpit: Storage pool → *Disks that can be added* → **Add to pool** → type the disk's name (e.g. `/dev/sda`) → **Erase and add**.

From a terminal: `storage-add`, choose the disk's number, type its name to confirm.

EgOS partitions and formats the disk, then runs an **integrity test**: it writes 128 MB of random data, reads it back and compares it, checks for kernel errors and measures the speed. A "fake" disk (counterfeit capacity) or a faulty one is rejected and does **not** join the pool.

Emptying the system disk

After you add a disk, the files already there stay on the system disk (visible and working). To move them: **Empty system disk** in Cockpit, or `storage-drain`. The paths do not change and the services keep working during the

move. Files changed in the last 10 minutes are left for the next run. You can close the page: the operation carries on on the server.

Removing a disk

1. With the disk **still connected**: **Remove** on its row (or `storage-remove`).
2. Type the branch name (e.g. `data1`) to confirm. The disk is **not** formatted.
3. Wait for the message **YOU CAN NOW SAFELY UNPLUG THE DISK**, then unplug it.

If it was the last disk, new files go back to the system disk.

If a disk gets disconnected

At boot EgOS waits 60 seconds for the disks, then starts anyway in **degraded** mode:

- the missing disk's files are not visible;
- **Jellyfin stays off** on purpose: otherwise its library scan would delete the missing titles together with their metadata and watch history;
- Immich and the shared folder keep working;
- a red banner appears in Cockpit (Overview and Storage pool) and at SSH login.

Plug the disk back in: it is remounted **automatically** within seconds and Jellyfin restarts. Alternatively press **Retry missing disks** (`storage-reattach`). If the disk is gone for good, press **Remove** on its row.

Optional services

The image contains two extra services, **off** until you turn them on.

NAME	WHAT IT OFFERS	ADDRESSES
`qbtt`	qBittorrent + search page + automatic sorting of downloads into Films, SerieTv, Musics	`:8082` client, `:8083` search
`bento`	PDF tools in the browser (merge, split, compress…)	`:8080`

```
egos-extras list             # list and status
egos-extras enable qbtt      # the first time it stops: edit the .env it
names
egos-extras enable qbtt      # second time: starts
egos-extras disable qbtt     # stops (files and data stay)
```

For `qbtt`, at the first start follow the `README.md` in its folder: temporary password (`docker logs qbittorrent-nox`), Web UI credentials, "Bypass authentication for clients on localhost" and the command `python3 /scripts/qbt-sort.py "%K"` to run when a download completes. Torrents that contain executables or scripts are **not** moved and get a `POTENTIALLY HARMFUL` file.

Command quick reference

COMMAND	EFFECT
`storage-status`	pool and disk status
`storage-add`	add a disk (erases it)
`storage-drain`	move files from the system disk to the data disks
`storage-remove`	remove a disk safely
`storage-reattach`	remount disks that were plugged back in
`egos-extras list`	optional services
`cat /etc/egos-release`	installed version
`cd ~/app/repos && docker compose ps`	container status
`cd ~/app/repos && docker compose restart`	restart the stack
`journalctl -u egos-pool`	pool log
`journalctl -u docker-compose-startup`	log of the 2nd-boot preparation

Troubleshooting

SYMPTOM	LIKELY CAUSE	FIX
I cannot see the SetupBox network	the box has no Wi-Fi	use the cable: `http://setupbox.local` or the IP
`setupbox.local` / `egocenter.local` **does not open**	the device does not support mDNS	use the IP address (box monitor or router)
After setup Jellyfin/ Immich do not answer	download in progress or no Internet	wait; check `journalctl -u docker-compose-startup`; reboot: the preparation retries
Jellyfin does not start, red banner	a pool disk is missing	plug it back in (cable, power) or **Retry missing disks**
Another storage operation is running	another disk operation is in progress	wait for it to end and retry
***UNMOUNT REFUSED* while removing**	something is using the disk	**do not unplug it**; `cd ~/app/repos && docker compose down`, repeat, then `docker compose up -d`
FAILED: the data read back does not match	counterfeit or faulty disk	do not use it
Copy fails with "read-only file system" (EROFS)	every data disk has less than 20 GB free: by design the system disk no longer takes new files	free some space or add a disk; it does not happen with large, non-full disks
Cockpit: *Administrative access needed*	session with limited access	**Limited access** at the top right → password
`egos-extras enable` **stops at once**	`.env` still contains `CHANGEME`	edit the file it names and run it again
Pool "not mounted"	pool service stopped	**Retry missing disks**; `systemctl status egos-pool`

PART II

Administrator guide

Architecture

EgOS is produced by **two self-contained scripts** that need no external files:

SCRIPT	OUTPUT	BUILD ENVIRONMENT
`EgOS4-arm64.sh`	ready-made image for the Pi 5	throwaway `debian:trixie` container + mmdebstrap + qemu-user (`builder-arm64-raspPi/`), or WSL2 Debian 13
`EgOS4-amd64.sh`	ISO with automatic installer	throwaway `debian:trixie` container + live-build (`builder-amd64/`)

The shared parts (services, webapp, stack, disk management, Cockpit page) are **byte-for-byte identical** in the two scripts.

```
1st BOOT
  egos-sshkeys ........ unique SSH keys per box
  egos-growroot ....... (Pi only) partition grown
  egos-pool ........... mergerfs pool on /srv/egos/local
  first-boot-setup
    ↳ setup-webapp.sh ↳ webapp.py :80 ↳ user ↳ reboot

2nd BOOT
  egos-pool ↳ docker, smbd
  docker-compose-startup
    ↳ deploy-compose.sh ↳ dirtree.sh ↳ wizardall2.sh
  egos-pool-guard ..... Jellyfin off if a disk is missing

ALWAYS
  udev "egos-data*" ↳ egos-reattach
    ↳ egos-storage _autoreattach
```

Building the images

amd64 (from any Linux with docker or podman): a throwaway container builds the ISO in its own filesystem and hands back only the ISO and the log.

```
builder-amd64/build-iso.sh                # -> ./out/egos-<version>-
amd64.iso
ENGINE=podman builder-amd64/build-iso.sh   # force podman
```

By hand, with podman:

```
mkdir -p ~/egos-amd64 && cd ~/egos-amd64        # the script only
sudo podman run -it --rm --privileged --network=host \
  -v "$PWD:/build:Z" -w /build debian:trixie bash
apt-get update && apt-get install -y live-build curl ca-certificates gnupg
bash EgOS4-amd64.sh 2>&1 | tee /build/build.log
```

To rebuild by hand, run `lb clean` first (not needed with `builder-amd64/`).

arm64 (from any Linux with docker or podman): the same throwaway-container approach. The container works in an ext4 loop file and registers the arm64 emulation (qemu, binfmt) only for the duration of the build.

```
builder-arm64-raspPi/build-img.sh          # -> ./out/egos-<version>-
arm64.img
```

By hand (WSL2 Debian 13, folder on the WSL ext4, never `/mnt/c`):

```
sudo apt-get install -y mmdebstrap arch-test qemu-user qemu-user-binfmt \
    e2fsprogs dosfstools mtools fdisk curl ca-certificates
sudo bash EgOS4-arm64.sh 2>&1 | tee build.log
KEEP_ROOTFS=1 sudo -E bash EgOS4-arm64.sh       # quick rebuild
```

Both scripts **stop with an error** if a check fails: apt key fingerprints, critical packages missing or scheduled for removal, a second network manager, SSH keys in the image, missing files, unpinned versions.

Versions

Every change that ends up in the image bumps `EGOS_VERSION` (in both scripts). The installed system records the build data in `/etc/egos-release`. Docker images are pinned: Immich `v3.2.2`, Jellyfin `12.1`, Valkey and Postgres by digest.

Where things are

PATH	CONTENTS
`/srv/egos/pool/media/`	Films, Musics, SerieTv, ROMs, downloads (Jellyfin, Samba, qbtt)
`/srv/egos/pool/immich/upload/`	Immich originals
`/srv/egos/local/`	pool branch on the system disk
`/srv/egos/disks/dataN/`	data disks (ext4, label `egos-dataN`)
`~/app/repos/`	`docker-compose.yml`, `.env`, wizard and JSON files
`~/app/repos/egos-home/`	home page: `home.py`, `docker-proxy.cfg`, `icons/`
`~/app/immich/postgres/`	Immich database (**always** on the system disk)
`~/app/jellyfin/{config,cache}/`	Jellyfin configuration
`/usr/local/share/stack/`	original files of the stack and the extras
`/var/log/egos-storage/job.log`	log of the last disk operation

State files

FILE	MEANING
`/var/lib/setup-data.json`	setup form data (holds the password in plain text)
`/var/lib/setup-complete`	setup done: the webapp does not start again
`/var/lib/docker-compose-done`	stack prepared: the deploy does not run again
`/var/lib/egos-growroot-done`	(Pi) partition already grown
`/run/egos/pool-degraded`	list of the missing disks
`/run/egos/pool-ok`	present only when the pool is complete (Jellyfin does not start without it)
`/var/lib/egos/jellyfin-held`	Jellyfin held stopped by EgOS

FILE	MEANING
`/run/egos/storage.lock`	lock for disk operations
`/etc/fstab.egos-prev`	fstab before the last change

To **redo** the 2nd-boot preparation: `sudo rm /var/lib/docker-compose-done && sudo systemctl enable docker-compose-startup` and reboot.

> **NOTE** — Since 4.2.2 `wizardall2.sh` can be re-run: once Jellyfin's startup wizard is complete it skips it (the `/Startup/*` APIs answer 401 then) instead of waiting forever.

Network

One manager only: **systemd-networkd** (DHCP on `eth*` / `en*`) with **systemd-resolved**. mDNS is handled by avahi, Windows discovery by wsdd2. Wi-Fi is used only for the first-boot access point.

PORT	SERVICE
22	SSH
80	setup webapp (1st boot only)
139, 445	Samba
2283	Immich
8000	home page (egos-home)
8096/tcp, 7358/udp	Jellyfin
9090	Cockpit
8080, 8082, 8083, 6881	optional extras

Security and known limits

What to know about the security of version 4.3.0 and about what EgOS does not handle yet.

ITEM	STATUS	WHAT TO DO
Fixed root password, the same on every box, written in the scripts (preseed / mmdebstrap hook)	**open**	after installing: `sudo passwd root` or `sudo passwd -l root`
Unique SSH keys per box	**solved** (4.0)	—
Web setup without authentication, AP with a public password	open	do the first boot in a trusted environment
`setup-data.json` **keeps the password in plain text**	open (`.env` already 0600)	—
One password for system, Samba, Immich, Jellyfin, Cockpit	open	change each service's password from its own interface
Security updates	partial: Docker repository kept (4.0), no `unattended-upgrades`	`sudo apt update && sudo apt upgrade` regularly
No firewall	open	restrict 22/445/9090 to the LAN
`DB_PASSWORD=password` **for the Immich database (not exposed on any port)**	open	—
SSH with password, `X11Forwarding yes`**, no fail2ban**	open	key-only access, `X11Forwarding no`
Samba/wsdd2/LLMNR also listening on the Docker bridges	open	`interfaces` + `bind interfaces only` in `smb.conf`

ITEM	STATUS	WHAT TO DO
Immich database backup	missing	schedule a `pg_dump` to the pool
fstab written atomically and validated	**solved** (4.0)	—
Home page reads Docker	**limited** (4.3): through a proxy on 127.0.0.1:23750 that allows only `GET /containers/json` (names, images, ports, labels; no environment variables, logs or commands)	—
Disk unplugged while the box is running	not handled	the mount stays "dead" until reattach or reboot

Release notes

VERSION	DATE	MAIN CHANGES
4.3.0	2026-11-03	home page **egos-home** on `:8000` in the base stack: one card per container, read from Docker through a proxy that allows only the container list
4.2.3	2026-10-29	qbt-sort warning file in English (`POTENTIALLY HARMFUL`)
4.2.2	2026-10-29	`wizardall2.sh` can be re-run (skips Jellyfin's startup wizard once completed)
4.2.1	2026-10-29	**automatic** reattach when a disk is plugged back in; operation lock; missing-disk warning on Cockpit Overview and at SSH login
4.2.0	2026-10-28	pool present from the first boot; `drain` ; 60 s wait and degraded boot; Jellyfin gate; immutable mount points
4.1.0	2026-10-28	Cockpit **Storage pool** page; non-interactive mode and jobs in `egos-storage`
4.0.0	2026-10-28	PC and Raspberry Pi editions aligned; versioning; sudo, SSH, Cockpit, mergerfs; networkd only; NTP; verified Docker keys; optional extras

PART III

Technical reference

Scripts and functions

Every script of the project, with its functions. This is the collection of the documents in the `docs/scripts/` folder: the same material is there, one file per script.

BUILD SCRIPTS

EgOS4-arm64.sh

Type	build script, self-contained
Path	`builder-arm64-raspPi/EgOS4-arm64.sh`
Output	`egos-<version>-arm64.img` : an **already installed** image (not an installer: the Pi 5 does not boot ISO/GRUB and d-i does not support it)
Hardware	Raspberry Pi 5 (8 GB) + NVMe
Base	Debian 13 "trixie" arm64, kernel `linux-image-rpi-v8` (4K pages)
Version	`EGOS_VERSION=4.3.0`, Immich `v3.2.2`, Jellyfin `12.1`
Role	**reference** for the shared parts: sections 5b–11 and the shared part of 11b are identical in `EgOS4-amd64.sh`

Build environment

Recommended: throwaway container from any Linux host with docker or podman, see builder.md.

```
builder-arm64-raspPi/build-img.sh      # from the repository root -> ./
out/egos-<version>-arm64.img
```

By hand: WSL2 Debian 13 (x86_64) with arm64 emulation through qemu.

```
mkdir -p ~/egos-arm64 && cd ~/egos-arm64      # this script only, on the
WSL ext4 (NEVER /mnt/c)
sudo apt-get update
sudo apt-get install -y mmdebstrap arch-test qemu-user qemu-user-binfmt \
    e2fsprogs dosfstools mtools fdisk curl ca-certificates
sudo bash EgOS4-arm64.sh 2>&1 | tee build.log
KEEP_ROOTFS=1 sudo -E bash EgOS4-arm64.sh     # reuses rootfs/ (skips
mmdebstrap, ~11 min)
```

Writing to the NVMe: from Windows with Raspberry Pi Imager → "Use custom" → `\\wsl.localhost\<distro>\home\<user>\egos-arm64\egos-4.3.0-arm64.img` (NVMe through a USB adapter), or `dd` from a native Linux. In the Pi 5 EEPROM `BOOT_ORDER` must include `6` (NVMe): `rpi-eeprom-config`.

Resulting flow

```
1st boot → egos-growroot (grows the partition) → first-boot-setup.service
           → webapp :80 → setup-data.json → user → REBOOT
2nd boot → time-sync + DNS → docker-compose-startup.service
           → dirtree.sh → wizardall2.sh
(egocenter.local)
```

Global variables

VARIABLE	MEANING
`W`	the script's folder
`ROOT` / `OVL` / `KEYS`	`rootfs/`, `overlay/`, `keys/`
`EGOS_VERSION`, `IMMICH_VERSION`, `JELLYFIN_VERSION`, `BUILD_DATE`	as in amd64
`IMG`	`egos-<version>-arm64.img`
`LABELID`	`45474f53` ("EGOS"): `PARTUUID=45474f53-01` (boot) and `-02` (root)
`BOOT_MB`	512
`KEEP_ROOTFS`	`1` = reuse the existing rootfs

Initial checks: run as root; the folder must be on ext2/3/4 (not DrvFs `/mnt/c`, which loses owners and permissions); commands `mmdebstrap arch-test mke2fs mkfs.vfat mcopy sfdisk curl gpg`.

Sections

#	SECTION	WHAT IT DOES
0	Emulation	if the host is not aarch64: mounts `binfmt_misc`, restarts `systemd-binfmt` (or runs it by hand if WSL has no systemd), checks with `arch-test arm64`
1	apt keys	Raspberry Pi key taken from pi-gen (re-signed SHA-512: the one from archive.raspberrypi.com uses SHA-1 signatures rejected by apt 3.x since 2026-02-01) with fingerprint `CF8A…303E`; Docker key with fingerprint `9DC8…CD88`. Also copied to the host
5b–11a	Shared	identical to amd64: `/etc/issue`, avahi, services, `setup-webapp.sh`, `webapp.py`, `deploy-compose.sh`, `dirtree.sh`, stack, `/etc/egos-release`, extras
11b	Pi-specific + shared	hostapd patch for `brcmfmac` (mandatory country code, rfkill), banner, `egos-storage` and units, Cockpit page, network, NTP, `egos-growroot`, `egos-sshkeys`, `config.txt`, `cmdline.txt`, `fstab`
12	rootfs	`mmdebstrap --variant=important` with the packages (below); hooks: copy the overlay, install cockpit without Recommends, purge `ifupdown dhcpcd-base dmidecode`, hostname `egocenter`, `/etc/hosts`, time zone `Europe/Rome`, locale `it_IT.UTF-8`, keyboard `it`, root password, `docker` group, enable the services, disable hostapd/dnsmasq, remove the SSH keys, `resolv.conf` → stub, empty `machine-id`. Then: Docker key in the final system (security updates), aliases in `/etc/skel/.bashrc`
13	Kernel	finds the `*-rpi-v8` kernel, temporarily disables the `z50-raspi-firmware` hook (it needs `/boot/firmware` mounted), generates the initramfs, copies `kernel8.img`, `initramfs8`, the `bcm2712*` DTBs and the overlays into `/boot/firmware`
14	Checks	critical packages (also `linux-image-rpi-v8`), no second network manager, release, pinned versions, no SSH keys, `egos-sshkeys` enabled, `resolv.conf` link, aliases, Cockpit page, reattach, pool units, stack on the pool
15	Image	**without loop devices**: `mkfs.vfat -C` for the boot partition (512 MB, `EGOSBOOT`) filled with `mcopy`; `mke2fs -d` for the root (`EGOSROOT`, size = rootfs × 1.3 + 512 MB); `sfdisk` with a DOS table and fixed label-id; `dd` of the two partitions into the image; label-id check

Packages (section 12)

`systemd-sysv systemd-resolved systemd-timesyncd dbus udev kmod locales keyboard-configuration console-setup tzdata initramfs-tools linux-image-rpi-v8 raspi-firmware firmware-brcm80211 wireless-regdb rfkill cloud-guest-utils e2fsprogs dosfstools zstd hostapd dnsmasq avahi-daemon python3 iptables iw curl ca-certificates samba wsdd2 sudo htop nano less lsof rsync openssh-server mergerfs parted attr net-tools progress git python3-requests docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin` + `cockpit-ws cockpit-bridge cockpit-system`.

Repositories: Debian trixie and trixie-security, `archive.raspberrypi.com`, Docker.

Boot files

FILE	CONTENTS
`/boot/firmware/config.txt`	`arm_64bit=1`, `kernel=kernel8.img` (4K pages: compatibility with arm64 Docker images that use jemalloc), `auto_initramfs=1`, `dtparam=pciex1` (NVMe HAT without HAT+ EEPROM)
`/boot/firmware/cmdline.txt`	`console=serial0,115200 console=tty1 root=PARTUUID=45474f53-02 rootfstype=ext4 fsck.repair=yes rootwait`
`/etc/fstab`	root `PARTUUID=…-02` ext4 `noatime`; boot `PARTUUID=…-01` vfat

Differences from amd64

	AMD64	ARM64
Output	ISO + installer	ready-made image
Build	throwaway `debian:trixie` container + live-build	throwaway `debian:trixie` container (or WSL2) + mmdebstrap + qemu-user
Docker	`.deb` files downloaded and installed locally	apt repository during mmdebstrap
First boot	—	`egos-growroot` grows the root
Hostname/locale	from the d-i preseed	from the mmdebstrap hooks

Functions

No functions of its own in the script; the function in `egos-growroot.sh` is described in egos-growroot.md.

EgOS4-amd64.sh

Type	build script, self-contained (no external files)
Path	`builder-amd64/EgOS4-amd64.sh`
Output	`egos-<version>-amd64.iso` (live-build `iso-hybrid`, d-i installer in "live" mode, unattended)
Base	Debian 13 "trixie" amd64
Version	`EGOS_VERSION=4.3.0`, Immich `v3.2.2`, Jellyfin `12.1`
Lines	~4150
Twin	EgOS4-arm64.md: sections 5b–11 and the shared part of 11b are **copied verbatim** from `EgOS4-arm64.sh`. They are edited there and then copied here.

Usage

Recommended: throwaway container, see builder.md.

```
builder-amd64/build-iso.sh      # from the repository root -> ./out/egos-
<version>-amd64.iso
```

By hand:

```
# on the Linux host with podman, folder holding ONLY this script
mkdir -p ~/egos-amd64 && cd ~/egos-amd64
sudo podman run -it --rm --privileged --network=host \
  -v "$PWD:/build:Z" -w /build debian:trixie bash
# inside the container
apt-get update && apt-get install -y live-build curl ca-certificates gnupg
bash EgOS4-amd64.sh 2>&1 | tee /build/build.log
```

To rebuild by hand: `lb clean` in the same folder before running again (not needed with `builder-amd64/`). At the end: `sudo chown $USER:$USER egos-*.iso`.

Resulting flow

```
USB → automatic d-i installation → power off (remove the USB stick)
1st boot → egos-sshkeys → first-boot-setup.service → webapp :80
(setupbox.local)
           → credentials → Linux user → REBOOT
2nd boot → time-sync + DNS → docker-compose-startup.service
           → dirtree.sh → wizardall2.sh
(egocenter.local)
```

Global variables

VARIABLE	MEANING
`EGOS_VERSION`	bumped on every change that ends up in the image
`IMMICH_VERSION`, `JELLYFIN_VERSION`	pinned Docker image tags
`BUILD_DATE`	UTC build date, written to `/etc/egos-release`
`ISO`	name of the final file
`OVL`	`config/includes.chroot`: root of the files copied into the system (same name as in the arm64 script)
`KEYS`	folder for the apt keys

First of all the script checks that `lb curl gpg apt-get` exist.

Sections

#	SECTION	WHAT IT DOES
1	`lb config`	trixie, amd64, d-i "live", areas `main contrib non-free non-free-firmware`, firmware in chroot and binary, hybrid ISO, volume `EGOS_4_2_3`
2	Docker key	downloads the key, converts it (`gpg --dearmor`) and **checks the fingerprint** `9DC8 5822 9FC7 DD38 854A E2D8 8D81 803C 0EBF CD88` (error 2 if missing). Puts it in the overlay and prepares `docker.list` in `/usr/local/share/egos/apt/` (copied into the system by `late_command`, not used during the build)

#	SECTION	WHAT IT DOES
3	Preseed	`config/includes.installer/preseed.cfg`, see below
4	Package list	`egos.list.chroot`: systemd-resolved/timesyncd, hostapd, dnsmasq, avahi, python3(+requests), iptables, iw, rfkill, wireless-regdb, curl, samba, wsdd2, sudo, openssh-server, htop, nano, less, lsof, rsync, mergerfs, parted, attr, net-tools, progress, git, zstd, cockpit-ws/bridge/system
4b	Docker .debs	adds the Docker repo **in the build container**, downloads `docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin` into `config/packages.chroot/` (renames `%3a` to `.`)
4c	Chroot hook `0400-egos`	see hook-live-build.md
5	Binary hook `0500-bootloader-default`	installer as the default entry, 5 s timeout, BIOS and UEFI
5b	Console and mDNS	`/etc/issue` (URLs and IP on the login screen), `avahi-daemon.conf` with the temporary name `setupbox`
6	Services	`first-boot-setup.service`, `docker-compose-startup.service`
7	`setup-webapp.sh`	see setup-webapp.md
8	`webapp.py`	see webapp-py.md
9	`deploy-compose.sh`	see deploy-compose.md
10	`dirtree.sh`	see dirtree.md
11	Stack staging	`/usr/local/share/stack/`: `docker-compose.yml`, `egos-home/` (since 4.3), `wizardall2.sh`, `jellsrv.json`, `jellremot.json`, `env.template` (with the versions substituted) + `/etc/egos-release`. See stack-configuration.md
11a	Optional extras	`/usr/local/share/stack/extras/{qbtt,bento}` + `egos-extras`. Not started. See egos-extras.md
11b	Shared overlays	hostapd patch (`country_code=IT`, `ieee80211d=1`, `rfkill unblock wlan`, checked with grep), extra lines in `/etc/issue`, `egos-storage`, aliases, pool/guard/reattach units, udev rule, empty `/etc/motd`, docker/smbd drop-ins, Cockpit page, networkd/resolved network, NTP wait, `egos-sshkeys.service`
12	Build	`lb build`
13	Checks	see below; then renames `live-image-amd64.hybrid.iso` to `$ISO`

Preseed (section 3)

AREA	SETTING
Language/keyboard	`it_IT.UTF-8`, keyboard `it` pc105
Network	automatic interface, hostname `egocenter`, domain `lan` (not `local`: that is the mDNS domain and would cause a false avahi conflict)
Mirror	`deb.debian.org`
Clock	UTC, `Europe/Rome`, **NTP on** (the TLS pulls of the 2nd boot need it)
Partitioning	`regular`, `atomic` recipe, no confirmation. With several disks d-i asks for the target disk
Accounts	root login enabled with a fixed password (see *Known issues*), no regular user (the webapp creates it)
Packages	no tasks, `full-upgrade` only: everything is already in the squashfs
Bootloader	GRUB on the default disk
`late_command`	removes `ifupdown dhcpcd-base isc-dhcp-client network-manager` and `/etc/network/interfaces`; enables networkd, resolved, timesyncd, time-wait-sync, avahi, wsdd2, docker, cockpit.socket, egos-sshkeys, first-boot-setup, docker-compose-startup, egos-pool, egos-pool-guard, ssh; disables hostapd/dnsmasq; creates the `docker` group; deletes the SSH host keys; copies `docker.list`; `resolv.conf` → resolved stub
End	no final message, **power off**

Final checks (section 13)

The build fails if:

- one of the critical packages is not installed in the chroot (`docker-ce containerd.io docker-compose-plugin samba wsdd2 hostapd dnsmasq avahi-daemon cockpit-ws sudo openssh-server htop rsync mergerfs parted attr git python3-requests systemd-resolved`);
- a second network manager is installed (`network-manager modemmanager wpasupplicant ppp ifupdown dhcpcd-base`);
- there are SSH host keys in the squashfs;

- `/etc/egos-release` is missing or has a different version;
- the aliases in `/etc/skel/.bashrc`, the Cockpit page files, the udev rule or `egos-reattach.service`, or the pool/guard units are missing;
- `env.template` does not point to the pool or has no pinned versions;
- **anti-purge**: a critical package appears in `binary/live/filesystem.packages-remove` (d-i would uninstall it at the end of the installation: it happened with cockpit installed by a hook).

Functions defined in the script itself

The main script defines no functions of its own: it is a linear sequence of sections. The functions live in the scripts it generates (see the other documents in this folder).

Builders — building the ISO and the Pi image in throwaway containers

	AMD64	ARM64 (RASPBERRY PI 5)
Folder	`builder-amd64/`	`builder-arm64-raspPi/`
Build script	`EgOS4-amd64.sh`	`EgOS4-arm64.sh`
Host command	`build-iso.sh`	`build-img.sh`
Container image	`egos-builder`: `debian:trixie` + `live-build curl ca-certificates gnupg`	`egos-builder-arm64`: `debian:trixie` + `mmdebstrap arch-test qemu-user qemu-user-binfmt e2fsprogs dosfstools mtools fdisk curl ca-certificates gnupg util-linux`
Output	`OUT/egos-<version>-amd64.iso` + `OUT/build.log`	`OUT/egos-<version>-arm64.img` + `OUT/build.log`
Time	~6 min	~10 min (arm64 emulated)
Requirements	Linux with docker (daemon running) or podman; `sudo`	the same + the host's `loop` module, ~30 GB of sparse space in the engine's storage

Usage

```
# from the repository root
builder-amd64/build-iso.sh                  # -> ./out/egos-<version>-
amd64.iso
builder-arm64-raspPi/build-img.sh ~/img     # output folder of your choice
ENGINE=podman builder-amd64/build-iso.sh    # force podman (default:
docker if the daemon answers)
SCRIPT=/other/EgOS4-amd64.sh builder-amd64/build-iso.sh
```

Each host command stops if the output folder already holds an ISO/image, so as not to overwrite it.

How it works (both)

1. `docker build --network=host` (or `podman build`) creates the builder image from the folder's `Dockerfile`. It holds only the tools; it is rebuilt from the layer cache. `.dockerignore` keeps the build script out of the image.
2. `docker run --rm --privileged --network=host` starts a throwaway container with: - the build script mounted **read-only** at `/src`; - the output folder mounted at `/out` (the only one shared with the host).
3. Inside the container `build-in-container.sh` copies the script to `/build`, runs the build, copies **only** the image and the log to `/out` and gives them to the host user.
4. When the build ends the container is deleted: no chroot, rootfs, cache or `binary/` is left on the host (about 5 GB with the manual method), so **no** `lb clean` or `rm -rf rootfs` is needed between builds.

arm64 specifics (`builder-arm64-raspPi/build-in-container.sh`)

- **ext4 work folder**: `EgOS4-arm64.sh` refuses any work folder that is not ext2/3/4 (it guards against WSL's DrvFs `/mnt/c`), and the container's own filesystem is overlayfs. The build therefore runs in a 30 GB sparse `/work.ext4` file, attached to a free loop device and mounted on `/build`. `/dev` in a privileged container is a snapshot taken at start, so if the kernel hands out a new loop device the script creates its node with `mknod`. On exit: unmount, `losetup -d`, file deleted.

- **arm64 emulation**: binfmt_misc belongs to the host kernel. If `qemu-aarch64` is not registered, the script registers it from the container's `/usr/lib/binfmt.d/qemu-aarch64.conf` (flags `OPF`: the interpreter is opened at registration time, so it works inside mmdebstrap's chroot) and **removes it on exit**. If it was already registered on the host, it is left alone. Nothing is installed on the host.
- The build itself is the unchanged `EgOS4-arm64.sh`: section 0 finds the emulation working (`arch-test arm64`).

Notes

- Both need **root** + `--privileged`: live-build and mmdebstrap mount `/proc` and create devices in the chroot; the arm64 builder also needs loop devices and binfmt_misc. A rootless engine is not enough.
- Every build downloads the packages again (~1 GB amd64, more for arm64).
- Both build and run use `--network=host`: on some hosts the engine's bridge network cannot reach the internet (firewall/VPN dropping forwarded traffic, docker rules lost, DNS fallback blocked) and the build stops at the first `apt-get update` of the Dockerfile. With the host network the builder needs only what the host itself can reach.
- Verified on 2026-10-29: amd64 with podman and Docker 29.9.2 (ISO installed in a VM up to the setup page); arm64 with Docker 29.9.2 (all section-14 checks, image contents inspected read-only; not boot-tested, there is no Pi 5 emulation).
- The amd64 ISO also includes the udebs of the previous installer kernel (~33 MB more than the manual build with a cache): the installer uses only those of its own kernel.

live-build hooks (amd64 only)

Generated by `EgOS4-amd64.sh` in `config/hooks/normal/`. live-build runs them during `lb build`.

0400-egos.hook.chroot

Section	4c
When	inside the chroot of the live system (= the installed system)
Shell	`/bin/sh`, `set -e`, `DEBIAN_FRONTEND=noninteractive`

Steps:

1. purges `ifupdown dhcpcd-base isc-dhcp-client network-manager` if installed (only systemd-networkd may manage the network);
2. deletes `/etc/ssh/ssh_host_*` (each box generates its own at the 1st boot with `egos-sshkeys`);
3. adds to `/etc/skel/.bashrc` the loading of `/etc/profile.d/egos-storage.sh`: Cockpit's terminal starts a **non-login** bash, which does not read `/etc/profile.d`. The user created at setup copies `.bashrc` from `/etc/skel`.

> **Never** `apt-get install` in this hook: packages added by a hook are not in `chroot.packages.install`, so live-build puts them in `filesystem.packages-remove` and d-i uninstalls them at the end of the installation (section 13 checks for this).

0500-bootloader-default.hook.binary

Section	5
When	"binary" stage (cwd = `binary/`, with a `cd binary` fallback)

Makes the installer the default entry with a 5-second timeout, based on the **actual** files generated by live-build trixie. Every assumption is checked: if it does not match, the build fails and prints the file.

FIRMWARE	ACTIONS	CHECK
BIOS (isolinux)	removes `menu default` from `live.cfg`, adds it after `label installstart` in `install.cfg`, `timeout 50` (tenths of a second) in `isolinux.cfg`	`label installstart` exists, `timeout 50` present

FIRMWARE	ACTIONS	CHECK
UEFI (GRUB)	appends `set default="Start installer"` and `set timeout=5` to `grub.cfg` (the last `set` wins)	`menuentry 'Start installer'` present in `install_start.cfg` (grep independent of the quotes)

Messages: `0500 BIOS ok: default=installstart, timeout 5s` and `0500 UEFI ok: …`; errors `ERROR 0500: …`.

FIRST AND SECOND BOOT

setup-webapp.sh

Path	`/usr/local/bin/setup-webapp.sh`
Generated by	section 7 of `EgOS4-amd64.sh` / `EgOS4-arm64.sh` (quoted heredoc: `$` is not expanded at build time) + the 11b patch
Started by	`first-boot-setup.service`, only if `/var/lib/setup-complete` is missing
When	1st boot, once

Purpose

Prepares the initial setup environment, waits for the user to fill in the web form, creates the Linux user, cleans everything up and reboots.

Steps

#	BLOCK	DETAIL
1	Interface detection	`WIFI` = first `wl*` / `wlan*` interface, `ETH` = first `en*` / `eth*` in `/sys/class/net`
2	**Wi-Fi access point** (only if there is a Wi-Fi card)	`rfkill unblock wlan` (11b patch), IP `192.168.50.1/24`; writes `/etc/hostapd/hostapd.conf`: SSID `SetupBox`, channel 7, 802.11g, WPA2-PSK, passphrase `setup1234`, `country_code=IT`, `ieee80211d=1` (11b patch); starts hostapd
3	Portal DHCP/DNS	`/etc/dnsmasq.d/setup.conf`: DHCP `192.168.50.10-50`, gateway and DNS = the box, `address=/#/192.168.50.1` (**every name** resolves to the box → phones and PCs open the "Sign in to network" popup by themselves), `setupbox.local`; starts dnsmasq
4	Redirection	iptables `PREROUTING`: incoming TCP 80 and 443 from Wi-Fi → `192.168.50.1:80`
5	Ethernet	`ip link set $ETH up`
6	mDNS	`/etc/avahi/services/http.service` (`_http._tcp` port 80, name `SetupBox`), restarts avahi: reachable as `http://setupbox.local`
7	Webapp	starts `/usr/local/bin/webapp.py` in the background
8	Wait	loops every 2 s until `/var/lib/setup-data.json` appears, then stops the webapp

#	BLOCK	DETAIL
9	User	reads `user`, `password`, `email`; `useradd -m -s /bin/bash`, `chpasswd`, groups `sudo` (or `wheel`) and `docker`; writes the email to `~/.email`
10	Cleanup	stops hostapd/dnsmasq, deletes their configuration and `http.service`; comments out `host-name=` in `avahi-daemon.conf` (from the 2nd boot the mDNS name becomes `egocenter.local`) and restarts avahi
11	End	creates `/var/lib/setup-complete`, disables `first-boot-setup.service`, `systemctl reboot`

Functions

None: a sequential script.

Files touched

FILE	ACTION
`/etc/hostapd/hostapd.conf`, `/etc/dnsmasq.d/setup.conf`, `/etc/avahi/services/http.service`	created and then deleted
`/etc/avahi/avahi-daemon.conf`	`host-name=` commented out
`/var/lib/setup-data.json`	created by the webapp, **kept** (used by dirtree, egos-storage, egos-extras)
`/var/lib/setup-complete`	end-of-setup flag
`/home/<user>/.email`	the email entered

Security notes

The form is plain HTTP, unauthenticated, on every interface; the AP passphrase is public and `setup-data.json` holds the password in plain text.

webapp.py

Path	`/usr/local/bin/webapp.py`
Generated by	section 8 of the EgOS4 scripts
Language	Python 3, standard library only (`http.server`)

Started by	`setup-webapp.sh`, 1st boot
Listens on	`0.0.0.0:80`

Purpose

Shows the "System Setup" form (Email, Username, Password) and saves the submitted data to `/var/lib/setup-data.json`.

Elements

`HTML` (constant)

Dark responsive page with the `POST /` form and the hint "Connect to **SetupBox** WiFi or visit **http://setupbox.local**".

class `Handler(BaseHTTPRequestHandler)`

METHOD	BEHAVIOUR
`log_message(format, *args)`	empty: no request logging
`do_GET()`	any path → 200 with `HTML` (needed for the captive portal: the Android/Apple/Windows probes get the page)
`do_POST()`	reads `Content-Length` bytes, `parse_qs`, builds `{"email","user","password"}` (empty string when missing), writes it to `/var/lib/setup-data.json`, answers "Setup Complete — The system will now reboot and deploy the stack."

Startup

`HTTPServer(('0.0.0.0', 80), Handler).serve_forever()`.

Format of `setup-data.json`

```
{"email": "you@example.com", "user": "username", "password": "…"}
```

Limits

No validation of the username (`useradd` validates it), no password confirmation, no CSRF protection or authentication: whoever submits the form first becomes the administrator.

egos-growroot.sh (arm64 only)

Path	`/usr/local/bin/egos-growroot.sh`
Generated by	section 11b of `EgOS4-arm64.sh`
Started by	`egos-growroot.service` (`Before=first-boot-setup.service`, condition: `/var/lib/egos-growroot-done` missing)
When	1st boot, once

Purpose

The `.img` image is small (rootfs × 1.3 + 512 MB); at the first boot the root partition is grown to fill the whole NVMe.

Steps

1. `SRC` = device of `/`; `PART` = its name; `DISK` = parent disk (`lsblk PKNAME`); `NUM` = partition number from `/sys/class/block/<part>/partition`.
2. `growpart $DISK $NUM` (exit code 1 = "NOCHANGE", not an error: `|| true`).
3. `resize2fs $SRC` (online).
4. Creates `/var/lib/egos-growroot-done` and disables the service.

Functions

None.

deploy-compose.sh

Path	`/usr/local/bin/deploy-compose.sh`
Generated by	section 9 of the EgOS4 scripts
Started by	`docker-compose-startup.service`
When	2nd boot (after `/var/lib/setup-complete`, as long as `/var/lib/docker-compose-done` is missing)

Purpose

Entry point of the stack deployment.

Steps

1. `set -e`; if `/var/lib/setup-data.json` is missing → "No setup data", exit 1.
2. Runs `/usr/local/bin/dirtree.sh`, which in turn `exec`s `wizardall2.sh`.
3. Only if everything succeeded: creates `/var/lib/docker-compose-done` and disables the service.

If the wizard fails, `set -e` stops the script **before** the flag: the service retries at the next boot.

Unit that starts it

`docker-compose-startup.service`:

- `After=network-online.target docker.service`, `Requires=docker.service`;
- drop-in `10-timesync.conf`: `After/Wants=time-sync.target` (correct clock before the TLS pulls);
- `ExecStartPre=-timeout 120 … until getent hosts deb.debian.org` (waits for DNS up to 120 s, the `-` ignores the timeout);
- `Type=oneshot`, `RemainAfterExit=yes`.

Functions

None.

dirtree.sh

Path	`/usr/local/bin/dirtree.sh`
Generated by	section 10 of the EgOS4 scripts
Started by	`deploy-compose.sh`, 2nd boot
Runs at the end	`exec ./wizardall2.sh`

Purpose

Creates the stack folders, copies the files from staging, generates credentials and `.env`, configures Samba and hands over to the wizard.

Variables

VARIABLE	VALUE
`STAGE`	`/usr/local/share/stack`
`U`	user read from `/var/lib/setup-data.json`
`H`	`/home/$U` (must exist, otherwise exit 1)
`POOL`	`/srv/egos/pool` (must be mounted, otherwise exit 1: the folders would end up under an empty mount point)

Steps

1. **Folders on the pool**: `media/{Films,Musics,SerieTv,ROMs,downloads}` and `immich/upload`, owned by the user.
2. **Folders in the home** (system disk): `~/app/repos`, `~/app/jellyfin/{config,cache}`, `~/app/immich/postgres`.
3. **Copy** (not `mv`: if the service fails and runs again, the sources are still there) of `docker-compose.yml`, `wizardall2.sh`, `jellsrv.json`,

`jellremot.json` into `~/app/repos`, and the `egos-home/` folder (`cp -rT`, so a retry copies over it instead of nesting it).

4. **Credentials** (Python block, `json.dump` handles the escaping): - `immichset.json` = `{"email", "password", "name": user}` → Immich administrator; - `jelluser.json` = `{"Name": user, "Password"}` → Jellyfin administrator. They are not included in the ISO: this way test credentials never end up in the image.
5. `.env`: `env.template` with `__HOME__` replaced by the absolute path (compose does not expand `$HOME` in `.env` files).
6. `chmod +x wizardall2.sh`, `chown -R` of `~/app`.
7. **Samba**: writes `/etc/samba/smb.conf` with the `[media]` share on `/srv/egos/pool/media`, setup user only, SMB2 minimum, no guests; `smbpasswd -a` with the same password; avahi service `_smb._tcp` (`EgoCenter`); restarts smbd and avahi, then wsdd2 (it has `BindsTo` on smbd, so it is restarted separately; `|| true` because network discovery is cosmetic).
8. `cd ~/app/repos && exec ./wizardall2.sh`.

Every step is idempotent: running them again at the next boot does no harm.

Functions

None.

wizardall2.sh

Path	`/usr/local/share/stack/wizardall2.sh` → copied to `~/app/repos/wizardall2.sh`
Generated by	section 11 of the EgOS4 scripts
Started by	`dirtree.sh` (`exec`)
Philosophy	"zero-errors mentality": `set -e`, every failing `curl -f` stops everything

Purpose

Starts the Docker stack and completes the initial configuration of Immich and Jellyfin through their APIs, so the user never has to open their setup wizards.

Steps

#	MESSAGE	ACTION
0	—	`cd` into the script's folder (the `curl -d @file.json` calls use relative paths)
1	`Starting compose`	`docker compose -f docker-compose.yml up -d`
2	`Waiting for Jellyfin…`	loops until `GET :8096/health` = 200
3	`Waiting for Immich…`	loops until `GET :2283/api/server/ping` contains `pong`
4	`IMMICH USERSET`	`POST :2283/api/auth/admin-sign-up` with `immichset.json`. If it fails (admin already created in an earlier attempt) it carries on
5	—	reads `GET /System/Info/Public` (retried until it answers); if it holds `"StartupWizardCompleted":true` the whole Jellyfin Startup block (steps 6–11) is skipped: "already completed in a previous run"
6	—	waits until `GET :8096/Startup/Configuration` = 200
7	`JELL SRVSET`	`POST /Startup/Configuration` with `jellsrv.json` (server name `EgoFin`, interface and metadata in Italian)
8	—	waits for `GET /Startup/User` = 200 (the GET is mandatory before the POST)
9	`JELL USERSET`	`POST /Startup/User` with `jelluser.json`
10	`JELL ACCESSET`	`POST /Startup/RemoteAccess` with `jellremot.json` (remote access yes, UPnP no)
11	`--- Complete ---`	`POST /Startup/Complete`
12	`--- Verify ---`	prints `GET /System/Info/Public`, then `ALL OK`

Functions

None.

Re-runs (since 4.2.2)

Once Jellyfin's wizard is complete, the `/Startup/*` APIs answer 401. Before 4.2.2 the wait in step 6 then never ended when `docker-compose-startup.service` retried at boot; now step 5 detects the completed wizard and skips it, so the script can be run again safely.

Stack configuration (compose, env, JSON)

Files staged in `/usr/local/share/stack/` by section 11 of the EgOS4 scripts and copied to `~/app/repos/` by dirtree.sh.

docker-compose.yml

Compose project `egos`, based on Immich's official compose file.

SERVICE	CONTAINER	IMAGE	PORTS	VOLUMES	NOTES
`immich-server`	`immich_server`	`ghcr.io/immich-app/immich-server:${IMMICH_VERSION}`	2283	`${UPLOAD_LOCATION}:/data`, localtime	`restart: always`, healthcheck
`immich-machine-learning`	`immich_machine_learning`	`…/immich-machine-learning:${IMMICH_VERSION}`	—	volume `model-cache`	for acceleration: suffix `-cuda`, `-openvino`, `-armnn` …
`redis`	`immich_redis`	`valkey/valkey:9` pinned by digest	—	—	healthcheck `redis-cli ping`

SERVICE	CONTAINER	IMAGE	PORTS	VOLUMES	NOTES
`database`	`immich_postgres`	`ghcr.io/immich-app/postgres:14-vectorchord0.4.3-pgvectors0.2.0` by digest	—	`${DB_DATA_LOCATION}`	`--data-checksums`, `shm_size 128mb`. **Local filesystem only**, never NFS/SMB
`docker-proxy`	`egos-home-docker-proxy`	`tecnativa/docker-socket-proxy:0.3.0` by digest	`127.0.0.1:23750`	`/var/run/docker.sock` (ro), `egos-home/docker-proxy.cfg` (ro, replaces the image's template)	since 4.3; allows only `GET /containers/json` and `/_ping`; `cap_drop: ALL`, `no-new-privileges`
`egos-home`	`egos-home`	`python:3.13-alpine` by digest	8000 (host network)	`egos-home/home.py`, `egos-home/icons` (ro)	since 4.3; user 1000, read-only, healthcheck `/health`. See egos-home.md
`jellyfin`	`jellyfin`	`jellyfin/jellyfin:${JELLYFIN_VERSION}`	8096/tcp, 7358/udp	config, cache, `Films`, `Musics`, `SerieTv→/Serie` (ro), `ROMs` (ro), **gate** `/run/egos/pool-ok→/egos-pool-ok` (ro, `create_host_path: false`)	`restart: unless-stopped`, `JELLYFIN_PublishedServerUrl=http://localhost:8096`, `host.docker.internal`

Restart policies: the four Immich services are `always`, Jellyfin `unless-stopped` (since 4.2: a Jellyfin held stopped by EgOS stays stopped across a reboot too). `JELLYFIN_PublishedServerUrl=http://localhost:8096` is not correct for clients connecting from other devices.

The **gate** stops Docker from starting Jellyfin while a pool disk is missing (see egos-storage.md). Without `create_host_path: false` compose would

pass the bind as a legacy `Binds` entry and Docker would create the missing source.

env.template → .env

KEY	VALUE	NOTES
`TZ`	`Europe/Rome`	
`UPLOAD_LOCATION`	`/srv/egos/pool/immich/upload`	Immich originals, on the pool
`DB_DATA_LOCATION`	`__HOME__/app/immich/postgres`	**system disk**
`DB_USERNAME` / `DB_PASSWORD` / `DB_DATABASE_NAME`	`postgres` / `password` / `immich`	default password, to be changed
`IMMICH_VERSION`	`v3.2.2`	substituted at build time for `@IMMICH_VERSION@`
`JELLYFIN_CONFIG_LOCATION`, `JELLYFIN_CACHE_LOCATION`	`__HOME__/app/jellyfin/{config,cache}`	system disk
`JELLYFIN_MEDIA_LOCATION`	`/srv/egos/pool/media`	on the pool
`JELLYFIN_VERSION`	`12.1`	

`__HOME__` is replaced by `dirtree.sh` with the absolute path of the home (compose does not expand `$HOME` in `.env` files).

Wizard JSON files

FILE	ORIGIN	CONTENTS	USED BY
`jellsrv.json`	staging	`ServerName: EgoFin`, `UICulture: it`, `MetadataCountryCode: IT`, `PreferredMetadataLanguage: it`	`POST /Startup/Configuration`
`jellremot.json`	staging	`EnableRemoteAccess: true`, `EnableAutomaticPortMapping: false`	`POST /Startup/RemoteAccess`
`immichset.json`	**generated** by dirtree	`email`, `password`, `name` = setup user	`POST /api/auth/admin-sign-up`
`jelluser.json`	**generated** by dirtree	`Name`, `Password` = setup user	`POST /Startup/User`

Resulting folder layout

```
/srv/egos/
├── pool/                  mergerfs (egos-pool.service)
│   ├── media/{Films,Musics,SerieTv,ROMs,downloads}   ← Jellyfin, Samba
[media], qbtt
│   └── immich/upload/                                ← Immich originals
├── local/                 branch on the system disk
└── disks/data1, data2…    one ext4 per data disk

/home/<user>/app/
├── repos/                 docker-compose.yml, .env, wizardall2.sh, *.json
│   └── egos-home/         home.py, docker-proxy.cfg, icons/
├── immich/postgres/       Immich database
├── jellyfin/{config,cache}/
├── qbtt/…                 (egos-extras enable qbtt)
└── pdf/bento/             (egos-extras enable bento)
```

egos-home — home page with one card per container

Path	`/usr/local/share/stack/egos-home/` → copied to `~/app/repos/egos-home/` by dirtree.sh
Files	`home.py` (server + page), `docker-proxy.cfg` (proxy rules), `icons/*.svg`
Generated by	section 11 of the EgOS4 scripts (heredocs `EGOSHOME_*_EOF`), since 4.3.0
Containers	`egos-home` (`python:3.13-alpine`) and `egos-home-docker-proxy` (`tecnativa/docker-socket-proxy:0.3.0`), both in the main `egos` compose project, see stack-configuration.md
Address	`http://egocenter.local:8000` (or the box's IP)
Language	Python 3, standard library only; page texts in Italian

Purpose

A launcher for the box: one card per service, with a link. It follows Docker by itself, so the optional stacks (`egos-extras enable qbtt`) and anything the user adds appear without editing a list.

How the cards are built

1. Every request reads `GET /containers/json?all=1` from the proxy (`DOCKER_API`, default `http://127.0.0.1:23750`).
2. **One card per container.** Port: label `egos.home.port`, otherwise the first published TCP port (bound to all addresses) that answers `HEAD /` as HTTP, then as HTTPS. The probe result is cached for 10 minutes. Databases, BitTorrent, Redis… do not speak HTTP and never get a card.
3. **Name, description, icon, order**: `egos.home.*` labels, otherwise defaults for known images (`KNOWN`: Immich, Jellyfin, qBittorrent, torrent-web, BentoPDF, Multi-scrobbler), otherwise the container name and image. Without `icons/<id>.svg` the card shows the initial on a colour derived from the name.
4. **State** from Docker: `on`; `warn` (running but `(unhealthy)`: yellow border, "non sano"); `off` (grey, not clickable). A stopped container keeps its card if it has a `port` label or was seen running since egos-home started.
5. **Static cards** (`STATIC`): services that are not containers — Cockpit (9090, https), checked by connecting to the port.
6. Links are built from the request's `Host` header (validated) + port; `egos.home.url` replaces the link (e.g. a public domain) and the card shows `↗` instead of the port.
7. If the proxy does not answer, a banner says so and only the static cards remain.

The page refreshes the states every 15 s through `/api/status` and reloads itself when containers appear or disappear.

Labels

LABEL	EFFECT
`egos.home.enable=false`	hides the container (set on `egos-home` and `docker-proxy`)
`egos.home.name`, `egos.home.desc`	card name and description

LABEL	EFFECT
`egos.home.port`, `egos.home.scheme`, `egos.home.path`	port, `http`/`https`, path
`egos.home.url`	full link instead of host:port
`egos.home.icon`	`icons/<value>.svg`
`egos.home.order`	sort key (default 50; Immich 10, Jellyfin 20, Cockpit 90)

Changing a label means recreating that container (`docker compose up -d` in its project).

Endpoints

PATH	ANSWER
`/`	the page
`/api/status`	`{"docker": bool, "services": {id: "on"/"warn"/"off"}}`
`/icons/<id>.svg`	icon (`[a-z0-9-]+` only, cached 1 day)
`/health`	`ok` (container healthcheck)

Functions (`home.py`)

FUNCTION	DESCRIPTION
`is_open(port)`	TCP connect to `127.0.0.1:port` (static cards)
`_speaks_http(port, tls)`	sends `HEAD /` (optionally over TLS without verification) and checks the answer starts with `HTTP/`
`web_scheme(port)`	`http`, `https` or `None`, cached `PROBE_TTL` (600 s)
`docker_containers()`	container list from the proxy, `None` if it does not answer
`known_for(image)`	defaults for a known image
`card_from_container(c)`	applies labels, port choice, defaults and state; `None` = no card
`services()`	containers (8 threads) + static cards, sorted by `order`, name
`request_host(header)`	host for the links, `FALLBACK_HOST` if the header is invalid
`icon_html(card)`	`<img>` of the icon or the generated initial

FUNCTION	DESCRIPTION
`render(host)` / `api_status()`	page / JSON
`Handler.do_GET`	routing; `send()` adds `no-store` and `nosniff`

Security

- The Docker socket goes **only** into the proxy. `docker-proxy.cfg` replaces the image's haproxy template and allows only `GET /containers/json` and `GET /_ping`; everything else answers 403. With the image's own `CONTAINERS=1` mode, `/containers/<id>/json` would expose every container's **environment variables** (passwords) and `/containers/<id>/logs` its logs to anything that can reach the proxy port.
- The proxy listens on `127.0.0.1:23750` (reachable by host processes and host-network containers), runs with `cap_drop: ALL` and `no-new-privileges`; it is not read-only because the image writes its `haproxy.cfg` at start.
- `egos-home`: read-only container, user 1000, host network (needed for the port probes); it has no authentication — it shows only links, on the LAN.

Risks and limits

- The stack now pulls two more images (`python:3.13-alpine`, `tecnativa/docker-socket-proxy`, both pinned by digest). As for the other images, a failed pull makes `docker compose up` fail and the deployment retries at the next boot.
- Port 8000 must be free on the box.
- Boxes installed before 4.3 do not get egos-home by themselves (no `egos-update` yet): the files and the two services have to be added to `~/app/repos` by hand.

STORAGE

egos-storage

Path on the system	`/usr/local/sbin/egos-storage`
Generated by	`EgOS4-amd64.sh` and `EgOS4-arm64.sh`, section 11b (heredoc `STORAGE_EOF`, identical in the two scripts)
Language	Bash (`set -euo pipefail`), with embedded Python 3 blocks
Run as	root only (`sudo`); otherwise it exits with "Run with sudo."
Version	EgOS 4.3.0

What it is for

It manages the EgOS **storage pool**: a mergerfs filesystem mounted on `/srv/egos/pool` that joins the folder `/srv/egos/local` (on the system disk) and the data disks `/srv/egos/disks/dataN` (one ext4 per disk, no RAID: if you lose a disk you lose its files, not the pool).

The pool exists **from the first boot**. The stack (Jellyfin, Immich, Samba) always points to `/srv/egos/pool/...`, so adding or removing a disk never changes a path.

SITUATION	POOL BRANCHES	WHERE NEW FILES GO
No data disk	`local=RW`	system disk
One or more data disks	`data1=RW:data2=RW…:local=NC`	data disks (`NC` = "no create": `local` stays readable/writable but takes no new files)
A disk missing at boot	only the disks present + `local`	pool "degraded", Jellyfin held stopped until the disk is back

> **NOTE** — With `minfreespace=20G` a branch with less than 20 GB free takes no new files. If every data disk is below that threshold (or the disk is smaller than ~20 GB) writes fail with EROFS, because `local` is `NC`. This is intended: the system disk no longer fills up.

Commands

Interactive (shell)

COMMAND	ALIAS (`/ETC/PROFILE.D/EGOS-STORAGE.SH`)	WHAT IT DOES
`egos-storage add` **(default)**	`storage-add`	formats a disk and joins it to the pool "hot"
`egos-storage status`	`storage-status`	pool and branch status
`egos-storage remove`	`storage-remove`	retires a branch (run it with the disk still connected)
`egos-storage reattach` / `remount`	`storage-reattach`, `storage-remount`	remounts disks that came back and refreshes the pool
`egos-storage drain`	`storage-drain`	moves files from the system disk to the data disks
`egos-storage link`	`storage-link`	installs older than 4.2 only: moves the stack data onto the pool

Non-interactive (Cockpit page, scripts)

COMMAND	OUTPUT
`status --json`	JSON with pool, branches, stack paths, job
`disks --json`	JSON of the disks that can be added
`add --disk DEV --confirm DEV`	the confirmation must repeat `DEV` exactly
`remove --branch dataN --confirm dataN`	the same with the branch name
`link --yes [--immich]`	`--immich` also moves the Immich originals
`job <command> <options>`	runs the command as the systemd unit `egos-storage-job` (survives a closed browser); log in `/var/log/egos-storage/job.log`

Internal (used by the systemd units)

COMMAND	CALLED BY
`_pool`	`egos-pool.service` (ExecStart)
`_pool-ready`	`egos-pool.service` (ExecStartPost)

COMMAND	CALLED BY
`_guard`	`egos-pool-guard.service`
`_autoreattach`	`egos-reattach.service`, started by the udev rule
`_job`	the transient unit created by `job`

Meaningful exit codes

CODE	MEANING
0	ok / cancelled by the user
1	generic error, wrong confirmation, lock busy
2	`add`: disk formatted and mounted but **not** joined to the pool
3	`reattach`: at least one disk is still missing

Variables and files

VARIABLE	VALUE	ROLE
`POOL`	`/srv/egos/pool`	mergerfs mount point
`LOCAL`	`/srv/egos/local`	branch on the system disk
`BRANCHES`	`/srv/egos/disks`	folder of the `dataN` mount points
`FSTAB`	`/etc/fstab`	managed disk lines (recognised by field 2)
`JOBUNIT` / `JOBLOG`	`egos-storage-job` / `/var/log/egos-storage/job.log`	long operations
`POOL_UNIT`	`egos-pool.service`	
`RUNDIR`	`/run/egos`	volatile state
`DEGRADED`	`/run/egos/pool-degraded`	list of missing disks (absent when the pool is complete)
`GATE`	`/run/egos/pool-ok`	exists only when the pool is complete; Jellyfin mounts it, so without it Docker cannot start Jellyfin
`HELD`	`/var/lib/egos/jellyfin-held`	persistent: "Jellyfin held stopped" survives a reboot

VARIABLE	VALUE	ROLE
`LOCK`	`/run/egos/storage.lock`	one change at a time
`MOTD`	`/etc/motd`	"disk missing" warning (Cockpit Overview + SSH login)
`DISK_WAIT`	`60` (can be overridden from the environment)	seconds to wait for slow USB disks at boot
`MFS_OPTS`	`allow_other,use_ino,cache.files=partial,dropcacheonclose=true,category.create=mfs,moveonenospc=true,minfreespace=20G,fsname=egospool,statfs_ignore=nc`	mergerfs options
`VERIFY_MB` / `VERIFY_TIMEOUT`	`128` / `180`	size and maximum time of the integrity test

Required commands (checked at start): `lsblk parted mkfs.ext4 blkid findmnt mountpoint mergerfs udevadm setfattr chattr rsync dd cmp timeout wipefs tr flock findfs`.

Functions

fstab handling

Every rewrite goes through a temporary file: NUL/control bytes are dropped and the result is checked before replacing the real fstab (an interrupted write can leave NUL bytes and an unmounted disk).

`fstab_commit <candidate_file>`

Validates and installs a candidate fstab.

1. `sync` of the temporary file.
2. `findmnt --verify --tab-file`: counts **only parse errors** (an unplugged `nofail` disk is an "error" for `--verify` but a legitimate state here).
3. Checks that the `/` line still exists.

4. If all is well: backup copy in `/etc/fstab.egos-prev`, atomic `mv`, `sync`. If it fails: prints the output, deletes the candidate, returns 1 and **does not touch** `/etc/fstab`.

`fstab_del <mountpoint>`

Removes the (uncommented) lines whose field 2 is `<mountpoint>`, stripping NULs; then `fstab_commit`.

`fstab_add <line>`

Appends a complete line at the end (after stripping NULs); then `fstab_commit`.

Concurrency

`storage_lock <seconds>`

Opens `/run/egos/storage.lock` on descriptor 9 and takes a `flock`, waiting at most `<seconds>` (0 = fail at once). It keeps the Cockpit job, a shell command and the udev reattach from overlapping (e.g. `add` formats a disk labelled `egos-dataN` and udev "sees" a pool disk appear). Taken with 0 seconds by `add`, `link`, `remove`, `reattach`, `remount`, `drain`; with 900 seconds by `_autoreattach`.

MOTD warning

`motd_update <missing_disks>`

Writes (or deletes, if the argument is empty) a block at the top of `/etc/motd` delimited by `*** EgOS STORAGE WARNING ...` `*** end of EgOS storage warning ***`. The block lists the missing disks and, if `HELD` exists, adds "Jellyfin is stopped until it is back". The rest of the file is preserved; the file is replaced only if the content changes.

The pool

> Golden rule (comment in the code): **never** unmount or restart the pool under a running stack. The containers would keep their bind mounts on the old, dead FUSE mount and see empty folders. Every change is applied hot through the branches extended attribute.

`pool_mounted`

True if `/srv/egos/pool` is mounted (`findmnt`).

`configured_disks`

Lists the `/srv/egos/disks/data*` mount points present in fstab, sorted `data1, data2, …` (`sort -V`).

`missing_disks`

Among the configured disks, prints those that are **not** mounted.

`branch_list`

Builds the branch string for mergerfs: the mounted data disks first (`=RW`), `local` last. `local` is `=RW` if no data disk is configured, `=NC` otherwise.

`update_state`

Updates the state in `/run/egos`: if disks are missing it writes the list to `pool-degraded` and removes the `pool-ok` gate; otherwise it deletes `pool-degraded` and creates `pool-ok`. Finally it calls `motd_update`.

`pool_start`

`systemctl start egos-pool.service` and checks that the pool is mounted. The mergerfs daemon must live in the service's cgroup, not the caller's: started from the Cockpit job it would be killed when the job ends.

`pool_sync`

Hot branch update. If the pool is not mounted, it starts it (`pool_start`) and calls `update_state`. Otherwise it writes `branch_list` into the `user.mergerfs.branches` attribute of the control file `/srv/egos/pool/.mergerfs` (`srcmounts` is read-only in Debian 13's mergerfs), then `update_state`. Returns 1 if the attribute cannot be written.

`mfs_branches <value>`

Writes the branches attribute directly; used by `do_remove` with `-/path` to detach a single branch.

`guard_dir <dir>` / `unguard_dir <dir>`

If `<dir>` is **not** a mount point, makes it immutable (`chattr +i`) / unlocks it (`chattr -i`). An empty mount point must never receive files: mounting over it works, writing into it does not.

The `egos-pool` service

`do_pool_run` (command `_pool`)

ExecStart of `egos-pool.service`.

1. Exits if the pool is already mounted.
2. **Migration**: installs older than 4.2 mounted the pool from fstab; if it finds that line it removes it (`fstab_del`) and runs `daemon-reload`.
3. Makes the pool mount point and those of the configured disks immutable (`guard_dir`).
4. Waits up to `DISK_WAIT` seconds (60) for missing disks to appear (slow USB).
5. `update_state`; if some disk is still missing it prints `DEGRADED: …`.
6. `exec mergerfs -f -o $MFS_OPTS <branches> /srv/egos/pool` (in the foreground: the process is the service).

`do_pool_ready` (command `_pool-ready`)

ExecStartPost: waits up to `DISK_WAIT + 30` seconds for the pool to show as mounted. This way the unit is "started" only once the pool is really there (docker and smbd start after it).

Protecting Jellyfin

Docker restarts its own containers at boot, before any EgOS unit can step in. That is why the compose file mounts the gate `/run/egos/pool-ok` inside Jellyfin with `create_host_path: false`: if the gate does not exist Docker **refuses** to start Jellyfin. This way the library scan never sees missing titles (it would delete them together with their metadata and watch history).

`jellyfin_release`

If `HELD` exists, runs `docker start jellyfin`; if that succeeds, deletes `HELD`.

`do_guard` **(command** `_guard`**)**

Run by `egos-pool-guard.service` after docker and the deployment.

- Degraded pool: waits up to 60 s for the `jellyfin` container to exist, creates `HELD`; if Jellyfin is running (stack installed before the gate existed) it stops it with `docker stop`, otherwise it records that the gate keeps it off. Updates the MOTD.
- Complete pool: `jellyfin_release`.

Reattach

`do_reattach` **(commands** `reattach`, `remount`**)**

For each missing disk it tries a plain `mount` (fails at once if the disk is absent, instead of waiting for the mount unit's timeout). Then `pool_sync`. If something is still missing it prints the disks and exits with code **3** ("Check cable and power... If the disk is gone for good: storage-remove"); otherwise `jellyfin_release`.

`do_autoreattach` **(command** `_autoreattach`**)**

Started by the udev rule `90-egos-reattach.rules` when a partition labelled `egos-data*` appears.

1. Does nothing if `egos-pool.service` is not active (at boot the service itself takes care of it).
2. `udevadm settle` (several disks plugged in together = a single pass), then `storage_lock 900`.
3. Looks, among the missing disks, for those whose fstab source (`UUID=…`) can now be found (`findfs`). It also considers the "stale" state: pool marked degraded but no disk missing any more.
4. If there is something to do it runs `do_reattach`, writing the output to `job.log` (visible in Cockpit as "Last operation") and to the journal.

Link (legacy installs)

`env_get <key>`

Reads the value of `<key>` from the stack's `.env` file (`$ENVF`), last occurrence.

`move_tree <env_key> <destination>`

Moves the folder named by a `.env` key onto the pool, with verification:

1. skips if the key is missing, already equal to the destination, or if the folder does not exist;
2. checks the free space (size + 1 GiB margin);
3. `rsync -aHAX --numeric-ids`;
4. **check 1**: a second `rsync` in dry-run must have nothing to transfer;
5. **check 2**: same number of files at source and destination;
6. renames the original to `<folder>.pre-pool` and updates `.env`. On any error the source stays intact and `.env` does not change.

`do_link` **(command** `link`**)**

For installs older than 4.2 (the stack was not on the pool). Finds the user from `/var/lib/setup-data.json`, asks (or takes from `--yes` / `--immich`) whether to move the Immich originals too, saves `.env` (`.env.bak.<date>`), stops the stack (`docker compose down`), calls `move_tree` for `JELLYFIN_MEDIA_LOCATION` → `/srv/egos/pool/media` and, if requested, `UPLOAD_LOCATION` → `/srv/egos/pool/immich/upload`, fixes the permissions, updates the `path` of the Samba share `[media]`, restarts smbd/wsdd2 and the stack. Immich's Postgres database **always stays on the system disk** (a DB on a USB disk is easily corrupted). The `.pre-pool` folders have to be deleted by hand.

Status

`do_status` **(command** `status`**)**

Prints: mounted branches (`findmnt`), each `dataN` with source/size/space or `NOT MOUNTED`, usage of `/srv/egos/local`; then the pool (`df`), the current

branch list read from the xattr, any `DEGRADED` state and "Jellyfin is stopped...".

`do_status_json` **(command** `status --json`**)**

Python block that produces the JSON read by the Cockpit page:

KEY	CONTENTS
`pool`	`path`, `mounted`, `size/avail/used`, `service` (state of egos-pool), `missing[]`, `degraded`
`branches[]`	for each `dataN`: `name`, `path`, `in_fstab`, `fstab_source`, `mounted`, `source`, `label`, space, `in_pool`; if not mounted, `stray` = files ended up on the system disk
`system_disk`	system disk (e.g. `/dev/nvme0n1`)
`local`	`path`, `in_pool`, `mode` (`RW`/`NC`), `has_files` (excluding the `.immich` markers), `system_free`
`jellyfin_held`	true if `HELD` exists
`stack`	`.env` path and values of `JELLYFIN_MEDIA_LOCATION`, `UPLOAD_LOCATION`, `DB_DATA_LOCATION`
`samba_media_path`	`path` of the `[media]` section of `smb.conf`
`job`	`unit`, `state`, `running`, `log`, `has_log`

Disk integrity test

`verify_disk <mountpoint>`

A fake-capacity device accepts writes, says "ok" and returns different data: `mkfs` succeeds, the ext4 gets corrupted later. The test:

1. generates 128 MiB of random data in `/var/tmp`;
2. writes it to the disk with `conv=fdatasync`, within 180 s (otherwise: disk unusable at this speed);
3. drops the cache (`drop_caches`) and compares byte by byte (`cmp`);
4. checks that the kernel logged no new `EXT4-fs error` in `dmesg`;
5. prints the approximate write speed. Returns 1 at the first failure.

Removal

`do_remove` (**command** `remove`)

Retires a branch **without formatting**. Run it with the disk still connected (it also works if it is already unplugged, but then any writes not yet on the disk are lost).

1. Lists the branches, picks the one from `--branch` or from numeric input.
2. Works out whether the disk is connected (`ATTACHED`); warns if it is the **last** data disk (new files go back to the system disk).
3. Asks to type the branch name exactly (`dataN`).
4. Detaches the branch hot (`mfs_branches -<path>`).
5. `sync`.
6. Unmounts: with the disk connected the unmount must succeed **cleanly**; if something keeps the branch open it shows `lsof`, puts the branch back in the pool and stops ("Do NOT unplug the disk"). With the disk absent it uses `umount -f`/`-l`.
7. Removes the line from fstab; if the empty folder remains, unlocks and removes it; if it is not empty it warns (files ended up on the system disk).
8. `pool_sync`; if no disk is missing any more, `jellyfin_release`.
9. Removes the device from the kernel (`/sys/block/<disk>/device/delete`) and prints **"YOU CAN NOW SAFELY UNPLUG THE DISK"**.

Adding

`sys_disk`

The disk holding `/` (e.g. `/dev/nvme0n1`), found with `findmnt` + `lsblk PKNAME`.

`candidates <system_disk>`

Python over `lsblk -J`: lists the whole disks that can be added, as TSV (`name size bus model mountpoint`). Excludes the system disk, `zram`, `loop`, `fd` (floppy controllers on x86 boards and VMs), eMMC boot/RPMB areas.

`do_disks_json` **(command** `disks --json`**)**

Converts the output of `candidates` into JSON (`name, size, tran, model, mountpoints`). Stops if the system disk cannot be determined.

`do_add` **(command** `add`**, default)**

1. Determines the system disk (if it cannot, it stops: it never risks formatting it).
2. Lists the candidates; picks from `--disk` or from numeric input.
3. Shows the contents; stops if there are mounted partitions.
4. Asks to type the device name exactly.
5. Picks the first free `dataN`: it **never reuses** the name of a configured but temporarily absent disk (its fstab line would be overwritten).
6. `wipefs -a`, GPT table, one partition at 100 %, `mkfs.ext4 -L egos-dataN -m 1`.
7. Creates the mount point (immutable), adds to fstab `UUID=… /srv/egos/disks/dataN ext4 defaults,nofail,noatime,x-systemd.device-timeout=60 0 2`, mounts.
8. `verify_disk`: if it fails it unmounts, removes the fstab line, removes the folder and stops ("the disk does NOT join the pool").
9. Gives ownership to the user created at setup (Samba and the services can write).
10. `pool_sync` (hot attach, no service restart). If it fails it exits with code **2** and suggests `storage-reattach`.

Drain

`do_drain` **(command** `drain`**)**

Moves files from `/srv/egos/local` to the data disks keeping the same relative path: no service notices.

1. Requires at least one mounted data disk and the pool mounted.
2. **Greedy plan** (Python): files from largest to smallest, each onto the disk with the most space, keeping 20 GiB in reserve per disk (like

`minfreespace`). Skips files changed in the last 10 minutes and the `.immich` markers. Prints the plan and the files that do not fit.

3. For each disk: `rsync -aHAX --from0 --files-from=… --remove-source-files` (rsync writes to a temporary name and renames at the end: nobody ever reads a half-written file).
4. The folder tree and the `.immich` markers always stay on the system disk: if every disk is removed the stack finds its folders again (Immich does not start without the markers).
5. Reports how many files are left; exits with 1 if some move failed (run it again to retry).

Jobs

`do_start_job <command…>` **(command `job`)**

Accepts only `add|remove|link|remount|reattach|drain`. Refuses if `egos-storage-job` is already active, then `systemd-run --unit=egos-storage-job --collect --setenv=HOME=/root … egos-storage _job <command…>`.

`do_run_job <command…>` **(command `_job`)**

Inside the unit: writes the header `== <date> egos-storage …` to `job.log`, runs the command with stdin from `/dev/null` appending everything to the log, closes with `== END rc=<code> <date>` and returns the code.

Final dispatcher

Takes `storage_lock 0` for the commands that change something, then routes `CMD` to the matching function; an unknown command prints the usage and exits with 1.

Related files and units

FILE	CONTENTS
`/etc/profile.d/egos-storage.sh`	`storage-*` aliases (also in `~/.bashrc` through `/etc/skel`, for Cockpit's terminal)
`/etc/systemd/system/egos-pool.service`	see systemd-udev-units.md

FILE	CONTENTS
`/etc/systemd/system/egos-pool-guard.service`	
`/etc/systemd/system/egos-reattach.service`	
`/etc/udev/rules.d/90-egos-reattach.rules`	
`/etc/systemd/system/{docker,smbd}.service.d/10-egos-pool.conf`	docker and Samba start after the pool
`/usr/share/cockpit/egos-storage/`	web interface, see cockpit-storage-pool.md

Cockpit page "Storage pool"

Path	`/usr/share/cockpit/egos-storage/` (`manifest.json`, `index.html`, `egos-storage.js`, `egos-storage.css`)
Generated by	section 11b of the EgOS4 scripts
Technology	plain HTML/JS + `../base1/cockpit.js` (Cockpit 337 ships no PatternFly in base1)
Access	`https://egocenter.local:9090` → menu **Storage pool** (position 35; keywords egos, pool, disk, mergerfs, storage, samba)

Principle

The page is only a front end for `egos-storage`: it reads the state with `egos-storage status --json` and `disks --json` and starts every change as `egos-storage job …` (a systemd unit). Closing the page **never interrupts** a format or a copy. All safety checks stay in `egos-storage`. Cockpit administrative access is required (`superuser: "require"`).

Page structure (`index.html`)

ELEMENT	CONTENTS
Header	title, explanation, **Refresh** button
`banner-admin`	"Administrative access needed": click *Limited access* at the top right

ELEMENT	CONTENTS
`banner-error`	read/start errors
`banner-health`	disk missing from the pool / pool not mounted
Pool card	status (badge), **Retry missing disks** button, usage bar, table of branches (Disk, Device, Size, Used, Free, State, Remove) with the system-disk row
Stack data card	**Empty system disk** (drain) and **Move data to pool** (legacy installs only) buttons, "Jellyfin is stopped" banner, media/photo/DB/Samba paths
Disks that can be added card	table of disks with an **Add to pool** button
Last operation card	job status and log
`<dialog id="dlg">`	confirmation dialog reused by every action

JavaScript functions (`egos-storage.js`)

Constants: `TOOL = /usr/local/sbin/egos-storage`, `POLL_IDLE = 10000` ms, `POLL_JOB = 2000` ms. State: `state` (last JSON), `timer`, `lastJobRunning`. `$ = document.getElementById`.

Utilities

FUNCTION	DESCRIPTION
`run(args)`	`cockpit.spawn([TOOL, …args], {superuser: "require"})`: runs egos-storage as root
`fmtBytes(n)`	formats bytes as B/KB/MB/GB/TB/PB (base 1000); `-` if absent
`el(tag, cls, text)`	creates a DOM element with class and text (always `textContent`, never HTML)
`badge(text, kind)`	creates a `<span class="badge kind">`
`setBadge(node, text, kind)`	updates an existing badge
`applyTheme(style)`	follows Cockpit's light/dark theme (`localStorage["shell:style"]`, `cockpit-style` event, `prefers-color-scheme`) by adding/removing the `dark` class

Rendering

FUNCTION	DESCRIPTION
`renderPool(st)`	pool badge (`not mounted` / `degraded` / `system disk only` / `ok`); usage bar computed **on the data disks only** when there are any (red ≥ 90 %); system-disk row (`new files` if RW, `old files only` if NC); one row per branch with status (`new files`, `not in pool`, `NOT MOUNTED`, "stray" files on the system disk) and a Remove button; `banner-health` if a branch is missing or not in the pool, or if the pool is not mounted
`renderStack(st)`	key/value list of the stack paths with badge `pool` / `system disk` / `pool NOT MOUNTED`
`renderDisks(all)`	table of the disks that can be added (excluding those already mounted under `/srv/egos/disks/`); if a disk has mounted partitions it shows "in use: unmount it first" instead of the button
`renderJob(st, log)`	job badge (`running`, `finished`, `failed (rc)`, `idle`) read from the `== END rc=` marker; cleans the `\r` of rsync/dd progress bars; disables the buttons during a job; shows **Move data to pool** only if the stack is not on the pool; enables **Empty system disk** only if there is a disk in the pool and files are left on the system disk; enables **Retry missing disks** only if there is a problem; shows `banner-held`

Loop and actions

FUNCTION	DESCRIPTION
`refresh()`	reads `status --json`, `disks --json` and the last 40,000 characters of the log; on access denied shows `banner-admin`, otherwise `banner-error`; then calls the renderers and reschedules itself every 2 s during a job, every 10 s when idle
`startJob(args)`	`egos-storage job …args`; on error shows the banner; scrolls to the log and refreshes
`dialog({title, body, okText, okKind, confirmWord, checkLabel})`	opens the modal and returns a Promise with `{checked, typed}` or `null`. If `confirmWord` is set the OK button stays disabled until exactly that word is typed
`askAdd(d)`	confirms "Erase and add" by typing the device name → `job add --disk D --confirm D`
`askRemove(b)`	warns (last disk, files no longer available), confirms by typing `dataN` → `job remove --branch dataN --confirm dataN`
`askLink()`	explains the copy and offers the "Also move the Immich photo originals" checkbox → `job link --yes [--immich]`

FUNCTION	DESCRIPTION
`askReattach()`	→ `job reattach`
`askDrain()`	→ `job drain`
`DOMContentLoaded`	applies the theme, wires the buttons, first `refresh()`, shows the body when the Cockpit transport is ready

CSS (`egos-storage.css`)

Styling in line with Cockpit (cards, `ok` / `danger` / `busy` badges, `warn` / `danger` banners, usage bar, tables, modal), with `.dark` variants and narrow-screen adaptation (the key/value list becomes a single column).

systemd units, drop-ins and udev rule

All generated by the EgOS4 scripts (identical on amd64 and arm64, except `egos-growroot`).

Boot order

```
1st boot:  egos-sshkeys ─┐
           (arm64) egos-growroot ─→ first-boot-setup ─→ setup-webapp.sh ─→
reboot
           egos-pool ─→ docker, smbd ─→ egos-pool-guard

2nd boot:  egos-pool ─→ docker ─→ docker-compose-startup (after time-sync
+ DNS) ─→ deploy-compose.sh
                                └→ egos-pool-guard

always:    udev "egos-data*" added ─→ egos-reattach ─→ egos-storage
_autoreattach
```

Units

UNIT	TYPE	ORDERING / CONDITIONS	RUNS
`first-boot-setup.service`	oneshot, RemainAfterExit	`After/Wants=network.target` ; only if `/var/lib/setup-complete` is **missing**	`/usr/local/bin/setup-webapp.sh`

UNIT	TYPE	ORDERING / CONDITIONS	RUNS
`docker-compose-startup.service`	oneshot, RemainAfterExit	`After=network-online.target docker.service`, `Requires=docker.service`; only if `setup-complete` **exists** and `docker-compose-done` is **missing**. Drop-in `10-timesync.conf`: `After/Wants=time-sync.target`. `ExecStartPre` waits for DNS up to 120 s	`/usr/local/bin/deploy-compose.sh`
`egos-sshkeys.service`	oneshot	`DefaultDependencies=no`, `After=local-fs.target`, `Before=ssh.service ssh.socket sockets.target`; only if `/etc/ssh/ssh_host_ed25519_key` is missing	`ssh-keygen -A`
`egos-growroot.service` **(arm64)**	oneshot	`Before=first-boot-setup.service`; only if `/var/lib/egos-growroot-done` is missing	`/usr/local/bin/egos-growroot.sh`
`egos-pool.service`	simple	`After=local-fs.target`, `Before=docker.service smbd.service egos-pool-guard.service`; `TimeoutStartSec=150`, `Restart=on-failure` (5 s)	`ExecStart=egos-storage _pool`, `ExecStartPost=egos-storage _pool-ready`, `ExecStop=-umount /srv/egos/pool`
`egos-pool-guard.service`	oneshot	`After=docker.service egos-pool.service docker-compose-startup.service`, `Wants=egos-pool.service`	`egos-storage _guard`
`egos-reattach.service`	oneshot, not enabled (udev starts it)	`After=egos-pool.service docker.service egos-pool-guard.service`; `TimeoutStartSec=1200`	`egos-storage _autoreattach`
`egos-storage-job`	transient (`systemd-run`)	created by `egos-storage job …`, `--collect`	`egos-storage _job …`

Drop-ins

FILE	CONTENTS	WHY
`/etc/systemd/system/docker.service.d/10-egos-pool.conf`	`Wants=` + `After=egos-pool.service`	containers restarted by the Docker daemon must find the pool mounted
`/etc/systemd/system/smbd.service.d/10-egos-pool.conf`	same	the `[media]` share is on the pool
`/etc/systemd/system/docker-compose-startup.service.d/10-timesync.conf`	`After/Wants=time-sync.target`	correct clock before the TLS pulls of the images

udev rule

`/etc/udev/rules.d/90-egos-reattach.rules`:

```
ACTION=="add", SUBSYSTEM=="block", ENV{DEVTYPE}=="partition",
ENV{ID_FS_LABEL}=="egos-data*", \
  TAG+="systemd", ENV{SYSTEMD_WANTS}+="egos-reattach.service"
```

udev must not run long jobs (`RUN+=` is killed after a few seconds): that is why it starts a service. At boot every present disk is "added": in that case `_autoreattach` does nothing.

Network configuration

FILE	CONTENTS
`/etc/systemd/network/20-wired.network`	`Name=eth* en*`, `DHCP=yes` (Wi-Fi is not managed by networkd: `setup-webapp.sh` configures it as an AP)
`/etc/systemd/resolved.conf.d/egos.conf`	`MulticastDNS=no`, `LLMNR=no` (mDNS is done by avahi on 5353, LLMNR by wsdd2 on 5355: no conflicts)
`/etc/resolv.conf`	link to `/run/systemd/resolve/stub-resolv.conf`
`/etc/avahi/avahi-daemon.conf`	`host-name=setupbox` (commented out at the end of setup), IPv4+IPv6, publishes addresses

Other system files

FILE	CONTENTS
`/etc/issue`	`EgOS 4.3.0 - Debian GNU/Linux 13`, URL `http://egocenter.local` and IP (`\4`), home page `http://egocenter.local:8000`, admin console `https://egocenter.local:9090`, storage and extras commands
`/etc/motd`	empty; `egos-storage` writes only the missing-disk warning there
`/etc/egos-release`	`EGOS_VERSION`, `EGOS_ARCH`, `EGOS_BUILD_DATE`, `EGOS_BUILD_SCRIPT`, `IMMICH_VERSION`, `JELLYFIN_VERSION`
`/etc/apt/sources.list.d/docker.list`	Docker repository with `signed-by=/etc/apt/keyrings/docker.gpg` (security updates)

OPTIONAL SERVICES

egos-extras

Path	`/usr/local/bin/egos-extras`
Generated by	section 11a of the EgOS4 scripts
Language	Bash (`set -euo pipefail`)
Stack sources	`/usr/local/share/stack/extras/<name>`

Purpose

Enables or disables the **optional stacks**, present in the image but **not started**:

NAME	CONTENTS	TARGET FOLDER	PORTS
`qbtt`	qBittorrent-nox + *torrent-web* search page + automatic sorting	`~/app/qbtt/qbittorrent-nox-easywebapp`	8082 (Web UI), 8083 (search), 6881 tcp/udp
`bento`	BentoPDF (PDF tools in the browser)	`~/app/pdf/bento`	8080

Usage

```
egos-extras list              available stacks and status
egos-extras enable <name>     copies into ~/app, creates .env, starts
egos-extras disable <name>    stops (files and data stay)
```

Initialisation

`U` = user from `/var/lib/setup-data.json` (must exist, otherwise "Setup user cannot be determined"), `H` = their home from `getent passwd`.

Functions

FUNCTION	DESCRIPTION
`dest_of <name>`	target folder of the stack; returns 1 for unknown names

FUNCTION	DESCRIPTION
`compose_of <name>`	compose file: `bento.yml` for bento, `docker-compose.yml` for the others
`media_dir`	value of `JELLYFIN_MEDIA_LOCATION` in the main stack's `.env`, or `/srv/egos/pool/media`
`as_user <paths…>`	when run as root, `chown -R` to the setup user
`dc <dir> <arguments…>`	`docker compose -f <compose_of $NAME> …` inside `<dir>`
`do_list`	for `qbtt` and `bento` prints name, status (`not installed` / `running` / `stopped`) and folder
`do_enable`	creates the folder, copies the files with `cp -r --update=none` (it **never overwrites** existing files); if there is an `env.example` and no `.env` it generates one replacing `__MEDIA__`, `__UID__`, `__GID__` (mode 600); for qbtt it creates `qbittorrent/config` and the folders `downloads, Films, Musics, SerieTv`; if `.env` still contains `CHANGEME` it stops and asks you to edit it; otherwise `up -d --build`; for qbtt it reminds you to follow the README
`do_disable`	`docker compose down` (if installed)

Dispatcher: `list` (default), `enable <name>`, `disable <name>`; otherwise it prints the usage and exits with 1.

Files of the `qbtt` stack

FILE	DESCRIPTION
`docker-compose.yml`	services `torrent-web` (local build, port `WEB_PORT`) and `qbittorrent-nox` (official image `qbittorrentofficial/qbittorrent-nox:${QBT_VERSION}`, `stop_grace_period: 30m`, `/tmp` on tmpfs, volumes config, scripts (ro), downloads, Films, Musics, SerieTv)
`dockerfile`	`python:3.13-slim`, non-root user uid 1000, TCP healthcheck on the port
`.dockerignore`	only `torrent_web.py` and `dockerfile` go into the image

FILE	DESCRIPTION
`README.md`	first-start instructions (temporary password from the logs, Web UI options, "Run on torrent finished" command)
`torrent_web.py`	see torrent_web.md
`scripts/qbt-sort.py`	see qbt-sort.md
`scripts/audiotags.py`	see audiotags.md
`env.example`	`WEB_PORT=8083`, `WEB_PASSWORD`, `QBIT_URL/USER/PASS`, `QBT_VERSION=5.2.4-1`, ports, `PUID/PGID`, paths

Files of the `bento` stack

`bento.yml`: service `bentopdf`, image `ghcr.io/alam00000/bentopdf-simple:latest`, port 8080, `DISABLE_IPV6=true`.

torrent_web.py

Path	`extras/qbtt/torrent_web.py` → container `torrent-web`
Generated by	section 11a of the EgOS4 scripts
Language	Python 3, standard library only
Port	`WEB_PORT` (default 8083)

Purpose

A small web page for the LAN: you paste one or more searches (one per line), the page runs them through qBittorrent's search engine, shows the results with filters (type, seeders, size, text) and adds them to the downloads with one click.

Configuration (environment variables)

VARIABLE	DEFAULT	MEANING
`QBIT_URL`	`http://localhost:8082`	address of the qBittorrent Web UI
`QBIT_USER`, `QBIT_PASS`	empty	Web UI credentials
`WEB_HOST`	`0.0.0.0`	listen address
`WEB_PORT`	`8083`	page port
`WEB_PASSWORD`	empty	if set, the page asks for user/password (Basic, any user)
`SEARCH_TIMEOUT`	60 s	(constant) maximum duration of a search

Classes and functions

`QbitError(Exception)`

Error with a message readable by the user.

`QBittorrent` — minimal Web API client

METHOD	DESCRIPTION
`__init__(url, user, password)`	stores the parameters, creates a lock and a session
`_new_session()`	urllib opener with a cookie jar and `Referer`/`Origin` headers (required by qBittorrent)
`_request(path, data, params, timeout)`	call to `/api/v2/<path>`; returns `(status, text)`; turns network errors into `QbitError` ("not reachable…")
`login()`	thread-safe; `auth/login`; accepts the `Ok.` body or checks with `app/version` (qBittorrent 5.2+ or localhost bypass); 403 = banned after too many attempts
`api(path, …)`	`login()` + request; on 403 logs in again once; ≥ 400 → `QbitError`
`enabled_plugins()`	names of the enabled search plugins
`search(text, category)`	`search/start` → waits for `Stopped` (max 60 s) → `search/results`; always `search/stop` and `search/delete`; normalises the results (`title`, `seeders`, `leechers`, `mb`, `site`, `page`, `link`) sorted by seeders

METHOD	DESCRIPTION
`add(link)`	`torrents/add` with the URL or the magnet

`QB` = global instance.

`PAGE` **(constant)**

HTML/CSS/JS page. Internal JS functions: `api(path, body)` (JSON fetch), `status(msg, error)`, `safeLink(u)` (accepts only `http(s)://`), `download(btn, link)`, `showGroup(query, results)` (renders a group of results).

`Handler(BaseHTTPRequestHandler)`

METHOD	DESCRIPTION
`log_message`	logs to stdout with the client address
`_authorized()`	if `WEB_PASSWORD` is set, checks the Basic header with `hmac.compare_digest`; otherwise 401 + `WWW-Authenticate`
`_send(code, body, ctype)`	JSON or text response, `Cache-Control: no-store`
`_json_body()`	accepts only `application/json` (blocks forms from other sites: CSRF) and at most 100,000 bytes
`do_GET()`	`/` or `/index.html` → page; `/api/plugins` → enabled plugins (502 if qBittorrent does not answer); anything else → 404
`do_POST()`	`/api/search` `{query, type}` → results; `/api/add` `{link}` (must start with `magnet:`, `http://`, `https://`) → adds it; errors 400/404/502

`main()`

Starts `ThreadingHTTPServer`, warns if `WEB_PASSWORD` is missing, shuts down cleanly on Ctrl+C.

qbt-sort.py

Path	`extras/qbtt/scripts/qbt-sort.py` → in the container `/scripts/qbt-sort.py` (read-only)
Generated by	section 11a of the EgOS4 scripts

Started by	qBittorrent, "Run external program on torrent finished": `python3 /scripts/qbt-sort.py "%K"`
Log	`/config/qbt-sort.log` (on the host: `qbittorrent/config/qbt-sort.log`)
Trial run	`docker exec qbittorrent-nox python3 /scripts/qbt-sort.py <hash> --dry-run`

Purpose

Sorts completed torrents into `/Films`, `/SerieTv` or `/Musics/<Artist>/<Album>/` (the layout Jellyfin expects). It uses the Web API (`torrents/setLocation`), so qBittorrent itself moves the files and seeding continues from the new location. It never makes qBittorrent fail: every exception is logged and the script exits with 1.

Constants

NAME	CONTENTS
`LOG`, `CONF`	log file and `qBittorrent.conf`
`API`	`http://127.0.0.1:<port>/api/v2/` (no login needed: "Bypass authentication for clients on localhost")
`DEST`	`video_movie → /Films`, `video_series → /SerieTv`, `audio → /Musics`
`EXT_VIDEO`, `EXT_AUDIO`	recognised extensions
`EXT_RISKY`	executables and scripts (`exe, msi, bat, ps1, sh, jar, apk, dmg, deb, …`)
`WARNING_FILE`	`POTENTIALLY HARMFUL` (Italian `POTENZIALMENTE DANNOSO` before 4.2.3)
`RE_SERIES`	`S01E02`, `S01`, `1x02`, `Season 1`/`Stagione 2`, `Complete Series`, `Ep 01` (season/stagione only when followed by a number: "The Four Seasons" is a film)
`RE_YEAR`, `RE_TAG`	title + year (takes the last one: "Blade.Runner.2049.2017"); cut at quality tags (`1080p`, `bluray`, `x265`, `ita`, …)
`RE_FEAT`, `RE_ARTIST_ALBUM`	removal of "feat./ft./featuring"; "Artist - Album" pattern
`UNKNOWN_ARTIST`, `VARIOUS_ARTISTS`	fallback values

Functions

FUNCTION	DESCRIPTION
`webui_port()`	Web UI port: from `QBT_WEBUI_PORT` or, since the image's `doas` clears the environment, from `WebUI\Port=` in `qBittorrent.conf`; default `8082`
`movie_folder(name)`	`'BlacKkKlansman.2018.1080p.BluRay'` → `'BlacKkKlansman (2018)'`: strips the video extension, extracts title and year (or cuts at the first tag), dots/underscores → spaces, removes invalid characters
`clean_name(name)`	makes a tag usable as a folder name (`AC/DC` → `AC-DC`), max 150 characters
`main_artist(artist)`	strips the "feat. ..." part
`names_from_title(name)`	fallback: `(artist, album)` from "Artist - Album (Year) [FLAC]"
`music_names(t, files)`	`(artist, album)` for an audio torrent: majority of the track tags (albumartist > artist; with ≥ 3 artists and no majority = "Various Artists"; album if ≥ 80 % agree), then the torrent name, then "Unknown Artist"/"Unknown Album"
`free_name(parent, name, own)`	`name`, or `name (2)`, `name (3)` ... if it already exists and is not the torrent's current location (existing albums are never merged)
`risky_files(files)`	list of files with a risky extension (also `song.mp3.exe`)
`mark_risky(name, risky, folder, dry)`	writes a `WARN` line to the log and, if the torrent has its own folder, creates the `POTENTIALLY HARMFUL` file there with the list
`log(msg)`	timestamped line on stdout and in the log file
`api(path, data)`	GET/POST to the local Web API
`torrent_info(hash_)`	torrent data (`torrents/info`) or `None`
`wait_for_move(hash_, max_seconds=3600)`	waits until the state is no longer `moving` (e.g. from the temporary folder)
`root_folder(files)`	name of the root folder if all files are in a single one

FUNCTION	DESCRIPTION
`classify(name, files)`	`audio` if the audio files weigh more; `video_series` if the name or ≥ 2 videos look like episodes; otherwise `video_movie`; `None` if there is neither audio nor video
`main()`	see below

Flow of `main()`

1. Arguments: `<hash>` and optional `--dry-run`.
2. `wait_for_move`; if the torrent does not exist → `ERROR`.
3. Reads the files (`torrents/files`).
4. **Risky files** → `mark_risky`, log `HOLD`, the torrent stays in `/downloads`.
5. `classify`; no audio/video → `SKIP`.
6. Works out destination and renaming: - single-file film → folder `Title (Year)`; - film already in a folder → renames the folder to `Title (Year)`; - audio → `/Musics/<Artist>/<Album>` (artist folder created if missing, album with `free_name`).
7. Already in the right place → `SKIP`; destination not mounted → `ERROR`.
8. `--dry-run` → log `DRY` and stop.
9. `torrents/renameFolder` if needed, `torrents/setLocation`, waits for the move to finish; if the location has not changed (e.g. permissions) → `ERROR`, otherwise `MOVE`.

Log prefixes: `MOVE`, `SKIP`, `HOLD`, `WARN`, `MKDIR`, `DRY`, `ERROR`.

audiotags.py

Path	`extras/qbtt/scripts/audiotags.py` (imported by `qbt-sort.py`)
Generated by	section 11a of the EgOS4 scripts
Language	Python 3, standard library only (the qBittorrent image has no `mutagen`)

Purpose

Minimal reader for the **albumartist**, **artist**, **album** tags. Formats: MP3 (ID3v2.2/2.3/2.4 + ID3v1), FLAC, Ogg Vorbis, Opus, M4A/MP4. Any other format → empty tags.

```
read_tags(path) -> {"albumartist": str|None, "artist": str|None, "album":
str|None}
```

Command-line use (debugging): `python3 audiotags.py file1.mp3 file2.flac` prints the name and tags.

Constants

NAME	CONTENTS
`KEYS`	`("albumartist", "artist", "album")`
`ID3_MAP`	ID3 frames: `TPE2/TP2` → albumartist, `TPE1/TP1` → artist, `TALB/TAL` → album
`VORBIS_MAP`	`ALBUMARTIST` / `ALBUM ARTIST` → albumartist, `ARTIST`, `ALBUM`
`MP4_MAP`, `MP4_CONTAINERS`	atoms `aART`, `©ART`, `©alb`; containers `moov udta meta ilst`

Functions

FUNCTION	DESCRIPTION
`_empty()`	dictionary with the three keys set to `None`
`_clean(s)`	NUL → `/`, strips spaces and `/` at the edges, empty string → `None`
`_syncsafe(b)`	decodes an ID3 "syncsafe" integer (7 bits per byte)
`_id3_text(data)`	decodes a text frame according to the encoding byte (0 Latin-1, 1 UTF-16, 2 UTF-16-BE, 3 UTF-8); for v2.4 multiple values (NUL-separated) keeps the first
`read_id3v2(f)`	reads the ID3v2 header at the start of the file, handles unsynchronisation (global in v2.2/2.3, per frame in v2.4) and the extended header, reads the mapped frames up to the padding; returns `(tags, tag_end)`
`read_id3v1(f)`	reads the final 128 `TAG` bytes (artist bytes 33–63, album 63–93)
`_vorbis_comments(b)`	parses a Vorbis comment block according to `VORBIS_MAP` (first value wins)

FUNCTION	DESCRIPTION
`read_flac(f, start)`	checks `fLaC`, walks the metadata blocks and reads the type-4 block (VORBIS_COMMENT)
`read_ogg(f, max_bytes=16 MiB)`	reassembles the 2nd logical packet from the `OggS` pages (it can span many pages if it holds the cover art) and reads it as a Vorbis (`\x03vorbis`) or Opus (`OpusTags`) comment
`read_mp4(f)`	walks the atoms with the inner function `walk(start, stop, depth)` (max depth 6, 64-bit atoms, `meta` with 4 version bytes) and reads the `data` child of the atoms in `MP4_MAP`
`read_tags(path)`	picks the reader from the extension: m4a/mp4/aac/alac/m4b → MP4; ogg/opus/oga → Ogg; otherwise ID3v2 (FLAC can also start with ID3) then FLAC, falling back to ID3v1 for the missing fields. I/O or format errors → empty tags